

CHAPTER

12

Fixed-point and Floating-point Arithmetic

Let us review how real numbers work. In general, a real number can often be written in the following format:

$$\pm d_0.d_1d_2d_3d_4d_5 \cdots d_{n-1} \cdots \times b^e$$

where b is the base (or radix) and each d_i is a digit ($0 \leq d_i < b$).

The decimal system, the most widely used number system in daily life, is based on powers of 10. Each digit represents a coefficient that multiplies the power of 10 represented by its position. For example,

$$654.321_{10} = 6 \times 10^2 + 5 \times 10^1 + 4 \times 10^0 + 3 \times 10^{-1} + 2 \times 10^{-2} + 1 \times 10^{-3}$$

All information is stored in the form of binary numbers in computers. A binary real number works in a similar way to a decimal real number.

$$101.011_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3}$$

If we convert it to the decimal notation, we get

$$101.011_2 = 4 + 0 + 1 + 0 + 0.25 + 0.125 = 5.375$$

There are two common ways to represent an approximation of a real number: fixed-point format and floating-point format. It is an approximation because a finite number of digits cannot represent all real numbers. For example, the actual value of $1/3$ is $0.333333\dots$, which has an infinite number of digits.

As introduced previously, the fixed-point format has a fixed number of digits after the decimal point. In contrast, the floating-point format can have a various number of digits after the decimal point depending on the scale of the real number.

While fixed-point arithmetic has fixed resolution for a given representation range and simple math allowing easy and fast computation, floating-point arithmetic trades manufacturing cost and computation efficiency for better precision and a wider range of representation. The processor often includes a special component, called floating-point unit (FPU), to speed up floating-point processing.

In embedded systems, not all microcontrollers have FPU on the chip. In the absence of an FPU, fixed-point arithmetic is often preferred, because the software implementation of floating-point arithmetic is much more complex than the fixed-point arithmetic.

Furthermore, if the dynamic range of an application's data set is small, the fixed-point format is preferred to improve the computational performance and accuracy. For example, video processing systems often use fixed-point arithmetic because pixel values have a fixed and regular format. On the contrary, audio systems often deploy floating-point arithmetic because the value of audio signals changes over a wide range.

While most desktops and servers use floating-point systems, the decision of using fixed-point or floating-point in an embedded system is sometimes difficult to make, as the cost of on-chip FPU has decreased sharply. For small applications running on limited computation capability or requiring long battery lifetime, fixed-point arithmetic is often preferred for simplified computation and improved energy efficiency. However, floating-point arithmetic can represent a wider dynamic range, and it is easier to develop a software program. The hardware has implemented complex floating-point functions, and the software can directly execute floating-point instructions supported by the hardware. Therefore, software programs do not need to worry about these functions.

"There is no sense in being precise when you do not even know what you are talking about."

John von Neumann
mathematician, computer
scientist



12.1 Fixed-point Arithmetic

As its name suggests, a fixed-point number assumes there are a predefined number of binary bits to the right of the binary point. Assembly programs treat a fixed-point number as a normal integer and use integer arithmetic to handle these numbers. However, for programmers, there is a virtual decimal place at a fixed location of the binary representation. When two fixed-point numbers are added or subtracted, the bit strings can be treated as two integers and can be directly added or subtracted because their

virtual decimal points are naturally aligned. However, when two fixed-point numbers are multiplied or divided, shift operations are required to fit the result into the same format as the operands.

12.1.1 Unsigned Fixed-point Representation

Unsigned fixed-point numbers often use the $UQ_{m.n}$ notation, where m is the number of bits representing the integer portion, and n is the number of bits representing the fractional portion, as shown in Figure 12-1. When m is 0, the notation is simplified to UQ_n , such as UQ_8 .

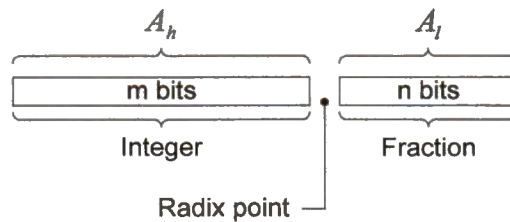


Figure 12-1. $UQ_{m.n}$ representation of unsigned fixed-point numbers

In the $UQ_{m.n}$ notation, the decimal value of an unsigned fixed-point number is calculated as follows:

$$f = A_h + A_l \times 2^{-n}$$

where A_h and A_l are the integer value of the integer portion and the fraction portion (h stands for high, l stands for low), respectively.

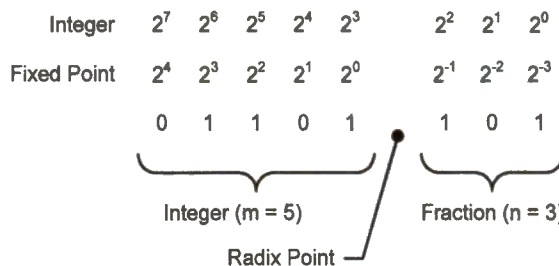


Figure 12-2. An example unsigned fixed-point number in $UQ_{5.3}$ format.

In the example given in Figure 12-2, the integer portion $A_h = 01101_2 = 13$, and the fraction portion $A_l = 101_2 = 5$. Therefore, we have

$$01101.101_2 = A_h + A_l \times 2^{-3} = 13 + 5 \times 2^{-3} = 13.625$$

Suppose I_A and f are the integer value and the unsigned fixed-point value that a sequence of binary digits represents, respectively. Then we have

$$f = \frac{I_A}{2^n}$$

For example,

$$01101.101_2 = \frac{1 \times 2^6 + 1 \times 2^5 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^0}{2^3} = \frac{109}{8} = 13.625$$

This concept is crucial because the computer usually implements fixed-point arithmetic by using integer operations. The key idea is that the computer processes these binary digits as if they were integers when in fact you, as the programmer, know that you are handling fixed-point numbers.

We can also use the radix or base to calculate the unsigned fixed-point value directly, as shown in the following example.

$$01101.101_2 = 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-3} = 13.625$$

12.1.2 Signed Fixed-point Representation

Signed fixed-point numbers often use the $Qm.n$ notation. The total number of bits N is $m + n + 1$, with m bits for the integer portion, n bits utilized for the fractional portion, and 1 bit for the sign. In digital signal processing, m is often 0, such as $Q0.7$ and $Q0.15$ (simplified as $Q7$ and $Q15$).

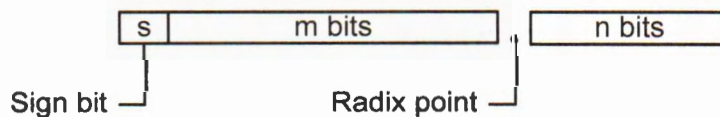


Figure 12-3. $Qm.n$ representation of fixed-point numbers ($m + n + 1$ bits totally)

Suppose binary digits $b_{N-1} \dots b_2 b_1 b_0$ represent a signed fixed-point number in the $Qm.n$ format. The signed integer value that these binary digits represent in two's complement is

$$A = -1 \times b_{N-1} \times 2^{N-1} + \sum_{i=0}^{N-2} (b_i \times 2^i) = -1 \times b_{m+n} \times 2^{m+n} + \sum_{i=0}^{m+n-1} (b_i \times 2^i)$$

where $N = m + n + 1$. Then the signed fixed-point value is

$$f = \frac{A}{2^n} = -1 \times b_{m+n} \times 2^m + \sum_{i=0}^{m+n-1} (b_i \times 2^{i-n})$$

Example: Find the Q4.3 signed fixed-point value that 10010011 represents.

$$10010.011_2 = \frac{-1 \times 2^7 + 1 \times 2^4 + 1 \times 2^1 + 1 \times 2^0}{2^3} = \frac{-109}{8} = -13.625$$

Like the unsigned fixed-point number, we often convert a signed fixed-point number f_A to an integer to facilitate the computation. In the above example, we have $I_A = 101010011_2 = -173$, because the binary integer is two's complement format.

12.1.3 Converting to Fixed-point Format

We can convert a real number to the fixed-point format $Qm.n$ or $UQm.n$ by multiplying it with 2^n and then rounding the product towards nearest integer A .

In the signed fixed-point format $Qm.n$, the integer A is represented in two's complement.

$$A = \text{round}(f \times 2^n)$$

Example 1: Convert $f = 3.141593$ to unsigned fixed-point UQ4.12 format.

- (1) Calculate $f \times 2^{12} = 12867.964928$
- (2) Round the result to an integer, $\text{round}(12867.964928) = 12868$
- (3) Convert the integer to binary: $12868 = 11_0010_0100_0100_2$
- (4) Organize into UQ4.12: $0011.0010_0100_0100_2$
- (5) Final result in Hex: $0x3244$
- (6) Error: $\frac{12868}{2^{12}} - f = -8.5625 \times 10^{-6}$

Example 2: Convert $f = -3.141593$ to signed fixed-point Q3.12 format.

- (1) Calculate $f \times 2^{12} = -12867.964928$
- (2) Round the result to an integer, $\text{round}(-12867.964928) = -12868$
- (3) Convert the absolute value to binary: $12868 = 11_0010_0100_0100_2$
Note that the integer is represented in two's complement.
- (4) Make the result into 16 bits: $0011_0010_0100_0100_2$
- (5) Find the two's complement: $1100_1101_1011_1100_2$
- (6) Final result in Hex: $0xCDBC$
- (7) Error: $-\frac{12868}{2^{12}} - f = 8.5625 \times 10^{-6}$

It is interesting that the signed and unsigned fixed-point representation in the above two examples are two's complement to each other.

12.1.4 Fixed-point Range and Resolution Tradeoff

An unsigned fixed-point number $UQ_{m,n}$ uses $m + n$ bits. For a signed fixed-point number $Q_{m,n}$, an extra sign bit is used and thus a total of $m + n + 1$ bits are used. Since the total number of bits available is given, the fixed-point representation must play a tradeoff between the range and resolution, depending on the application's needs.

The *resolution* is defined as the smallest non-zero real number representable. It equals the gap between two consecutive numbers that are representable in fixed-point format. For $UQ_{m,n}$ and $Q_{m,n}$, the resolution of n is 2^{-n} . For example, the resolution of $Q_{3,3}$ is 2^{-3} (i.e., 0.125).

Note resolution differs from accuracy.

- *Accuracy* refers to the closeness of a numeric representation of a number to its true value. For example, when representing a real number, such as π , in $UQ_{16,16}$, the error is the difference between the true value and the approximation in $UQ_{16,16}$. Naturally, the maximum error is less than the resolution 2^{-n} .
- *Resolution* describes precision. It is the smallest change that is representable in the digital system. The precision or resolution does not measure how close a representation is from its true value; instead, it measures how close the values of different representations in the same format can be from each other.

We define the representable range as the maximal and minimal values that can be represented.

- *Range of unsigned fixed-point numbers*
The range of unsigned integers represented by using $m + n$ bits is $[0, 2^{m+n} - 1]$. Therefore, the representable range of unsigned fixed-point numbers is $[0, 2^{m+n} - 1] \times 2^{-n}$, i.e., $[0, 2^m - 2^{-n}]$.
- *Range of signed fixed-point numbers*
The range of signed integers, represented in two's complement by using $m + n + 1$ bits, is $[-2^{m+n}, 2^{m+n} - 1]$. Therefore, the representable range of signed fixed-point numbers is $[-2^{m+n}, 2^{m+n} - 1] \times 2^{-n}$, i.e., $[-2^m, 2^m - 2^{-n}]$.

Figure 12-4 and Figure 12-5 shows the range and resolution of UQ_5 unsigned fixed-point numbers and Q_4 signed fixed-point numbers, respectively.

The fixed-point representation plays a tradeoff between range and resolution. If n is too small, then we have a poor resolution, but a large representation range. On the other hand, if n is too large, then we have a good resolution, but a small representation range and high risk of overflow. For unsigned fixed-point numbers, we can choose to use

UQ16.16. However, if we need a larger range, we may use UQ20.12; if a higher resolution is needed, we may use UQ12.20.

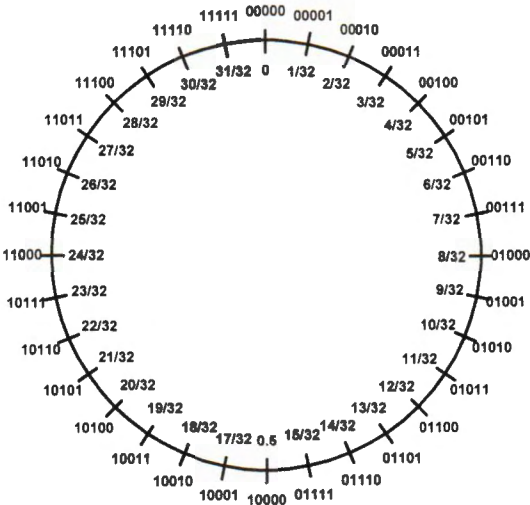


Figure 12-4. UQ5 unsigned fixed-point

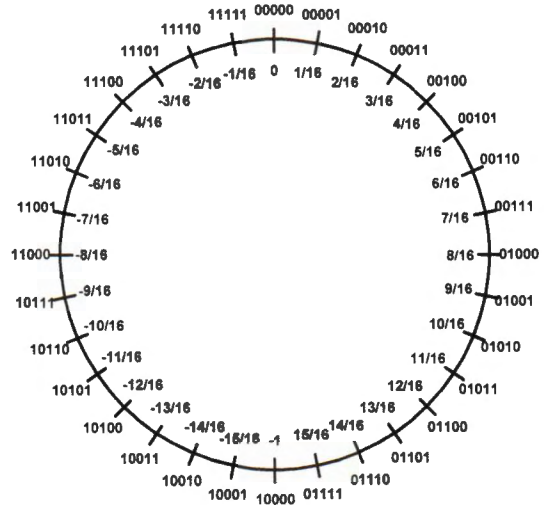


Figure 12-5. Q4 signed fixed-point

If the dynamic range of application's data set does not fit in the representable range of a given $Q_{m.n}$ format, we often use a predefined linear scaling and offsetting operation to transform the application's data set into the representation range. When the arithmetic operation completes, we perform a reverse scaling and shifting to recover the true data.

The linear scaling and offsetting are performed as follows. If the actual value of an application's data is V , and the represented approximation is $\tilde{V}$, then we have

$$\tilde{V} = V \times 2^{scale} + offset$$

where *scale* and *offset* are two constants (can be positive or negative value) predefined by the application to ensure that $\tilde{V}$ falls in the available fixed-point range.

12.1.5 Fixed-point Addition and Subtraction

Suppose f_A , f_B and f_C are three unsigned fixed-point numbers in the UQ16.16 format, or three signed fixed-point numbers in the Q15.16 format. We define f_C as:

$$f_C = f_A + f_B$$

The following shows how to calculate f_C by using integer arithmetic.

If the radix point (or binary point) is ignored, the integer value that is represented by the bit string of f_A , f_B and f_C is I_A , I_B , and I_C , respectively. For signed fixed-point numbers, I_A , I_B and I_C may be negative because each bit string denotes an integer in two's complement.

The following equations show the conversion between f_A, f_B, f_C and I_A, I_B, I_C .

$$\begin{cases} I_A = f_A \times 2^{16} \\ I_B = f_B \times 2^{16} \\ I_C = f_C \times 2^{16} \end{cases} \Leftrightarrow \begin{cases} f_A = I_A \times 2^{-16} \\ f_B = I_B \times 2^{-16} \\ f_C = I_C \times 2^{-16} \end{cases}$$

Thus, we can calculate the fixed-point addition $f_A + f_B$ as follows:

$$\begin{aligned} f_C &= f_A + f_B \\ &= I_A \times 2^{-16} + I_B \times 2^{-16} \\ &= (I_A + I_B) \times 2^{-16} \end{aligned}$$

Since

$$f_C = I_C \times 2^{-16}$$

Thus, we have the following equation:

$$I_C \times 2^{-16} = (I_A + I_B) \times 2^{-16}$$

We multiply 2^{16} on both sides of the above equation, and we have:

$$I_C = I_A + I_B$$

The above equation shows that f_C can be directly obtained by adding these two bit-strings that represent f_A and f_B . Adding two fixed-point numbers is performed in the same way as if these two bit-strings were representing two integers.

Similarly, for unsigned or signed fixed-point subtraction:

$$f_C = f_A - f_B$$

we have

$$I_C = I_A - I_B$$

Therefore, the binary result digits of fixed-point subtraction are the same as integer subtraction.

In sum, the addition and subtraction are simple. If bit strings represent fixed-point numbers, the program can treat them as integers during subtraction and addition. In other words, although they are fixed-point numbers, we can use integer “add” and “sub” instructions to calculate the addition and subtraction of two fixed-point numbers. The program tricks the processor into thinking adding or subtracting two integers while the program knows that fixed-point numbers are processed in fact.

```
; r0 = fixed-point number fa
; r1 = fixed-point number fb
; r2 = fixed-point number fc
add  r2, r0, r1      ; fc = fa + fb
sub  r2, r0, r1      ; fc = fa - fb
```

Example 12-1. Adding and subtracting two fixed-point numbers in UQ16.16 or Q15.16

12.1.6 Fixed-point Multiplication

This section shows how to calculate the fixed-point multiplication between f_A and f_B , i.e.

$$f_C = f_A \times f_B$$

where f_A , f_B and f_C are all unsigned fixed-point numbers in the UQ16.16 format or signed fixed-point numbers in the Q15.16 format. Mathematically, we have

$$\begin{aligned} f_C &= f_A \times f_B \\ &= (I_A \times 2^{-16}) \times (I_B \times 2^{-16}) \\ &= (I_A \times I_B) \times 2^{-32} \end{aligned}$$

We also know that

$$f_C = I_C \times 2^{-16}$$

Therefore, we get

$$I_C \times 2^{-16} = (I_A \times I_B) \times 2^{-32}$$

We can multiply 2^{16} on both sides of the above equation, and then we have

$$I_C = (I_A \times I_B) \times 2^{-16}$$

The above equation shows that we can treat fixed-point numbers f_A and f_B as integers first and calculate the product of these two integers. The result that we want is the middle 32 bits of the 64-bit product. Therefore, we need to shift the 64-bit product right by 16 bits, and the product is the least significant 32 bits of the shifted product.

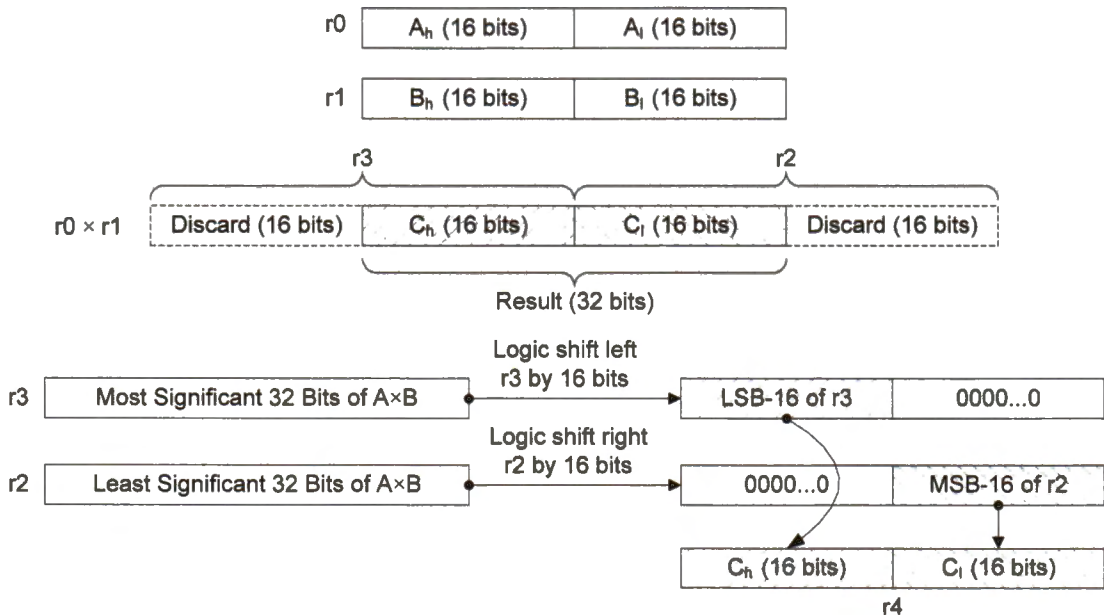


Figure 12-6. Multiplying two fixed-point numbers in UQ16.16 or Q15.16

Unsigned fixed-point format UQ16.16	Signed fixed-point format Q15.16
<pre> ; r0 = fixed-point number A ; r1 = fixed-point number B ; r4 = fixed-point product = A × B ; Unsigned long multiply ; r2 = low word, r3 = high word UMULL r2, r3, r0, r1 ; Shift left high word LSLS r3, r3, #16 ; Shift right low word LSRS r2, r2, #16 ; Pack two halfwords into a word ORR r4, r2, r3 </pre>	<pre> ; r0 = fixed-point number A ; r1 = fixed-point number B ; r4 = fixed-point product = A × B ; Signed long multiply ; r2 = low word, r3 = high word SMULL r2, r3, r0, r1 ; Shift left high word LSLS r3, r3, #16 ; Shift right low word LSRS r2, r2, #16 ; Pack two halfwords into a word ORR r4, r2, r3 </pre>

Example 12-2. Multiplying two fixed-point numbers in UQ16.16 or Q15.16

The multiplication should be unsigned for UQ16.16 and signed for Q15.16.

- Note that SMULL treats two source operands r0 and r1 as two's complement signed integers and produces a 64-bit product. If the exact product has 65 bits, we only keep the most significant 64 bits. SMULL places the least significant 32 bits of the 64-bit result into register r2 and the most significant 32 bits into register r3.
- For multiplying unsigned fixed-point numbers, we should use UMULL to multiply their corresponding integers.

12.1.7 Fixed-point Division

This section shows how to calculate the fixed-point division of f_A by f_B , i.e.

$$f_C = f_A \div f_B$$

where f_A , f_B and f_C are fixed-point numbers in UQ16.16 notation.

Mathematically we have,

$$\begin{aligned}
 f_C &= f_A \div f_B \\
 &= (I_A \times 2^{-16}) \div (I_B \times 2^{-16}) \\
 &= I_A \div I_B
 \end{aligned}$$

We also know that

$$f_C = I_C \times 2^{-16}$$

Therefore, we have

$$I_C = (I_A \div I_B) \times 2^{16} = (I_A \times 2^{16}) \div I_B$$

The above equation shows that we can treat fixed-point numbers f_A and f_B as integers first. Then we shift the dividend I_A left by 16 bits to make it a 64-bit number. After that, we divide this 64-bit number by the 32-bit divisor I_B . The 32-bit quotient is the result.

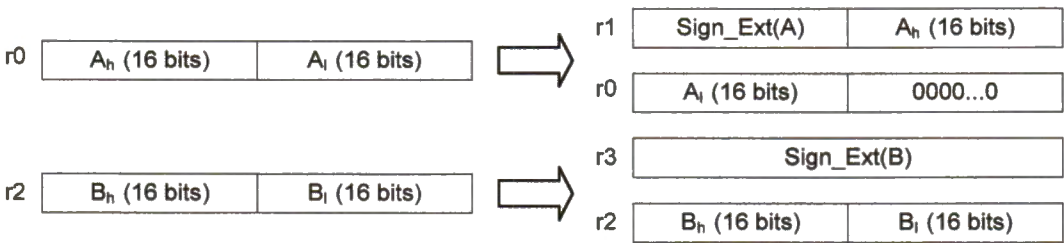


Figure 12-7. Dividing two fixed-point numbers in the format of UQ16.16 or Q15.16

```
; r0 = fixed-point number A
; r2 = fixed-point number B
; r4 = fixed-point quotient = A ÷ B

ASRS r1, r0, #16      ; 16-bit sign in upper word
LSLS r0, r0, #16      ; [r1:r0] stores sign extended A
ASRS r3, r2, #31      ; [r3:r2] stores sign extended B
BL   division_64_bits ; Take four input register (r0:r1, r3:r2)
MOV  r4, r0           ; division_64_bits places result in r0
```

Example 12-3. Dividing two fixed-point numbers in the format of UQ16.16 or Q15.16

ARM does not have any instructions that perform 64-bit integer division. Chapter 9.9 presents the implementation of 64-bit integer division.

12.2 Floating-point Arithmetic

The IEEE Standard for Floating-point Arithmetic (IEEE 754) is a *de facto* technical standard to store and operate real numbers in computers. It aims to provide portability across different platforms by defining a shared method of representing floating-point numbers and implementing floating-point operations. IEEE 754 has been widely supported by many processors with FPU and by many software libraries written for processors without FPU.

12.2.1 Floating-point Representation

The IEEE 754 standard uses a normalized notation, which includes three fields: the *sign* bit, the *fraction*, and the *true exponent*, as shown in Figure 12-7. We call it true exponent to differ from the biased exponent introduced later in this section.

The normalized notation implicitly assumes there is only one digit before the decimal point, and this digit must be 1. This bit is also called the *hidden bit* or the *hidden 1* in IEEE 754.

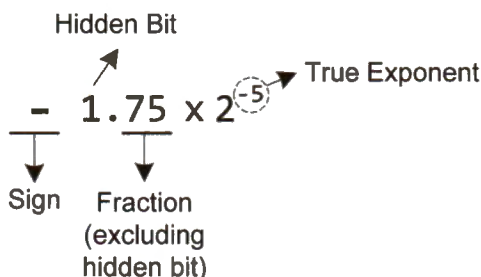


Figure 12-8. Normalized notation.

For example, the following numbers are not in normalized format: 10.746×2^6 , 5.023×2^3 , -0.5×2^{-7} , and 0.05×2^9 , because their integer part (all digits to the left of the decimal point) is not exactly 1. Additionally, 1.025×10^3 is not in the normalized format because the exponential part is based on 10, instead of 2.

We can convert 10.746×2^6 to the normalized format as follows:

$$10.746 \times 2^6 = \frac{10.746}{8} \times 8 \times 2^6 = 1.34325 \times 2^9$$

Single-precision and double-precision, defined in IEEE 754, are two of the most commonly used formats. Half-precision is often used in digital signal processing applications. The half-precision, single-precision, and double-precision use 16, 32, and 64 bits to store a floating-point number, respectively. In C, the statement "float x" declares a single-precision variable, and "double y" declares a double-precision variable.

Half Precision (16 bits)



Single Precision (32 bits)



Double Precision (64 bits)



Figure 12-9. IEEE 754 single-precision and double-precision format

Compared with single precision, double precision has higher accuracy and can represent a much wider range of numbers. However, double-precision takes more memory space and bandwidth and is less computationally efficient. While most scientific applications running on desktops or servers use double precision, many embedded systems use single precision to reduce the hardware cost and improve the energy efficiency.

Each format has three fields. As shown in Figure 12-9, the *half-precision* has a sign bit, a 5-bit biased exponent, and a 10-bit fraction. The *single-precision* has a sign bit, an 8-bit biased exponent, and a 23-bit fraction. The *double-precision* has a sign bit, an 11-bit biased exponent, and a 52-bit fraction. The following discusses these three fields.

- The *sign bit* indicates whether the number is positive or negative. A value of 0 in the sign bit stands for a positive number, whereas a value of 1 denotes a negative number.
- The *biased exponent* is defined as the sum of the actual exponent and a bias constant. In other words, the true exponent equals the biased exponent minus the bias constant. The bias constant is 15 for half precision, 127 for single precision, and 1023 for double precision. For single precision, the actual exponent has a value falling between -127 and 128, and, therefore, the biased exponent is always non-negative, ranging from 0 to 255. You may wonder why the exponent field does not use two's complement of the true exponent, instead of the biased exponent. The key reason is that comparing two unsigned numbers can be executed much faster than comparing two signed numbers represented in two's complement. This can speed up floating point comparison.
- The *fraction* field consists of all bits to the right of the binary point. In the normalized format, as shown in Figure 12-7, the leading bit (the bit to the left of the decimal point) is always 1. This hidden bit is therefore not stored in the fraction field. When calculating the decimal value of a binary format, the hidden bit should be included. For example, if the fraction field is 0b0100101 in binary, the actual fraction is 1.0100101 in binary, which includes the fraction bits and the hidden leading bit with a value of 1.

We can calculate the floating-point number represented in IEEE 754 format by using the following formula (except a few special numbers discussed later):

$$f = (-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent} - \text{Bias}}$$

where

$$\text{Bias} = \begin{cases} 15, & \text{Half Precision} \\ 127, & \text{Single Precision} \\ 1023, & \text{Double Precision} \end{cases}$$

The following gives two examples of converting floating-point numbers between different formats.

Example 1: Decoding 0xC1FF0000 into a floating-point number

We need to convert the hex value to a 32-bit binary bit string. These bits are divided into three groups: with bit 31 (the leftmost bit) being the sign bit, bits 30-23 (the next eight bits) being the exponent, and bits 22-0 (the rest) being the fraction.

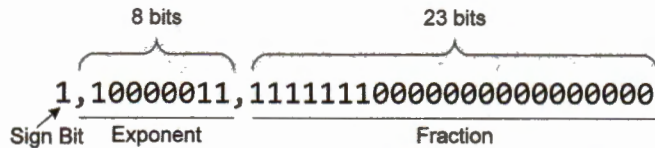


Figure 12-10. Partitioning bits of 0xC1FF0000 into three parts

1. The first step is to check the sign bit, which is the leading bit of the bit string. In this case, we have $S = 1$.
2. Convert the exponent `10000011` into a decimal integer.

$$10000011_2 = 131$$

3. Convert the fraction `111111100000000000000000` into a decimal real number.

$$\begin{aligned} 0.1111111_2 &= 1 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} + 1 \times 2^{-5} + 1 \times 2^{-6} \\ &\quad + 1 \times 2^{-7} \\ &= 0.5 + 0.25 + 0.125 + 0.0625 + 0.03125 + 0.015625 + 0.0078125 \\ &= 0.9921875 \end{aligned}$$

4. Calculate the represented real number as follows:

$$\begin{aligned} f &= (-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent} - 127} \\ &= (-1)^S \times (1 + 0.9921875) \times 2^{131 - 127} \\ &= -1 \times 1.9921875 \times 2^4 \\ &= -31.875 \end{aligned}$$

Example 2: Encoding 14.5 into single-precision IEEE 754 format

The encoding involves the following five steps:

1. First, set up the sign bit. The sign bit S is cleared for a positive number and is set for a negative number. Therefore, $S = 1$ in this case.

2. Rewrite the floating number in the base-2 normalized format. Since 14.5 is larger than 2, we can repeatedly divide it by 2 until the quotient is smaller than 2 but greater than 1. Note, if the floating-point number to be converted is smaller than 1, we multiply it by 2, instead of dividing it by 2.

$$\begin{aligned}14.5 \div 2 &= 7.25 \\7.25 \div 2 &= 3.625 \\3.625 \div 2 &= 1.8125\end{aligned}$$

Therefore, we have

$$\begin{aligned}14.5 &= 1.8125 \times 2^3 \\&= (1 + 0.8125) \times 2^3\end{aligned}$$

3. Calculate the exponent by adding the actual exponent and the bias. The bias is 127 for the single-precision.

$$\text{Exponent} = 3 + 127 = 130 = 1000010_2$$

4. Calculate the fraction by converting 0.8125 into binary. The conversion is achieved by repeatedly multiplying the fraction part (all digits to the right of the decimal point) of the product with 2 until the product becomes 1.

$$\begin{aligned}0.8125 \times 2 &= 1.625 = 1 + 0.625 \\0.625 \times 2 &= 1.25 = 1 + 0.25 \\0.25 \times 2 &= 0.5 = 0 + 0.5 \\0.5 \times 2 &= 1\end{aligned}$$

Thus, the binary representation of 0.8125 is 0.1101, which combines the leading digit of the products (the digit to the left of the decimal point) of the above multiplications. The following can help you understand why the above conversion process works.

$$\begin{aligned}0.8125 &= \frac{1}{2} \times (1 + 0.625) \\&= \frac{1}{2} \times \left(1 + \frac{1}{2} \times (1 + 0.25)\right) \\&= \frac{1}{2} \times \left(1 + \frac{1}{2} \times \left(1 + \frac{1}{2} \times (0 + 0.5)\right)\right) \\&= \frac{1}{2} \times \left(1 + \frac{1}{2} \times \left(1 + \frac{1}{2} \times \left(0 + \frac{1}{2} \times 1\right)\right)\right) \\&= 1 \times \left(\frac{1}{2}\right)^{-1} + 1 \times \left(\frac{1}{2}\right)^{-2} + 0 \times \left(\frac{1}{2}\right)^{-3} + 1 \times \left(\frac{1}{2}\right)^{-4} \\&= 2^{-1} + 2^{-2} + 2^{-4} \\&= 0.1101_2\end{aligned}$$

Thus, the fraction is 1101000000000000000000, which appends 0 to the result to make the length of the fraction bit string 23 bits.

5. The single-precision floating point format of 14.5 is 01000001011010000000000000000000 in binary and 0x41680000 in hex.

12.2.2 Special Values

Figure 12-11 shows bit patterns representing special values including 0, $\pm\infty$, and NaN.

- We cannot represent zero in IEEE 754 format directly. Instead, we represent zero with a special pattern in which all bits of exponent and fraction are cleared. Because it is not required to clear the sign bit, there is a positive zero and negative zero, and hardware should treat them as equal.
- We denote the positive infinity and the negative infinity by a special pattern in which all exponent bits are one, and all fraction bits are zero.
- NaN (Not any Number) means that a floating-point arithmetic is invalid. We represent a NaN by any particular patterns in which all exponent bits are one, and the fraction is non-zero. For example, the following operations produce NaN: $0.0/0.0$, $-\infty + \infty$, $0 \times (\pm\infty)$, $\pm\infty/\pm\infty$, $\text{sqrt}(-1.0)$, and $\log(-10.0)$. There are two types of NaN: QNaN (Quiet NaN) and SNaN (Signaling NaN). A SNaN generates a hardware exception signal to let the software handle the anomaly. A QNaN does not raise such a hardware signal.

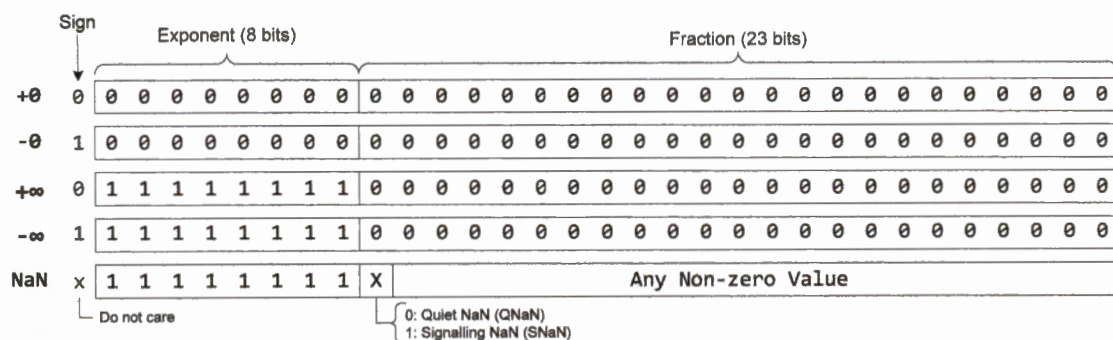


Figure 12-11. Special values in IEEE 754 single-precision

Floating operations normally use QNaN and let the NaN result propagate through most of all following arithmetic operations. For example, $0 \times \text{NaN} = \text{NaN}$, $\text{sqrt}(\text{NaN}) = \text{NaN}$, logic expressions " $0 < \text{NaN}$ ", " $0 > \text{NaN}$ ", and " $0 = \text{NaN}$ " are false, but " $0 \neq \text{NaN}$ " is true. Many processors, including ARM Cortex-M, are configurable so the same operation can produce either a QNaN or a SNaN to meet the application's need. In the default setting, QNaN is preferred over SNaN.

12.2.3 Overflow and Underflow

Now let us find the smallest and largest values that the single-precision format can represent.

- **Numbers closest to zero.** Since `0b00000000` is reserved in the exponent field, the minimal value of the exponent is `0b00000001`. The fraction can be as small as `0b000...00`. Thus, the numbers closest to zero are

$$(-1)^S \times (1 + 0) \times 2^{1-127} = \pm 2^{-126} \approx \pm 1.18 \times 10^{-38}$$

- **Numbers farthest from zero.** The maximum value of the exponent is `0b11111110` because `0b11111111` is reserved. The fraction can be as large as `0b111...111`. Thus, the biggest finite values are

$$(-1)^S \times (1 + (1 - 2^{-23})) \times 2^{254-127} = \pm(2^{128} - 2^{104}) \approx \pm 3.40 \times 10^{38}$$

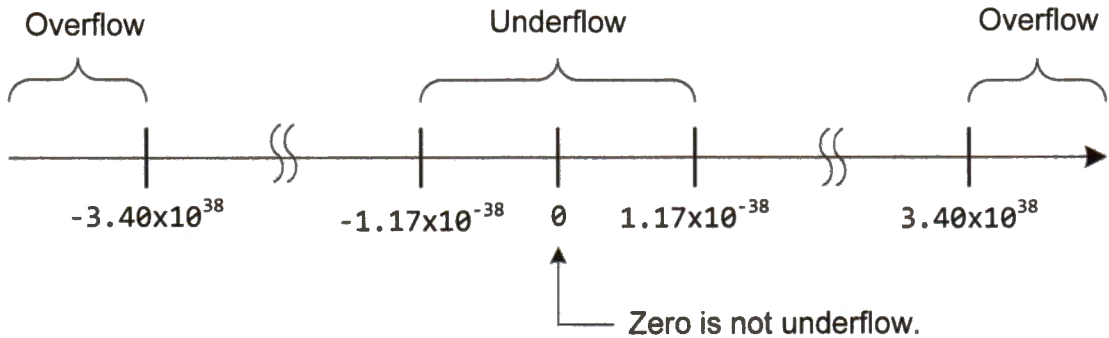


Figure 12-12. Representation range of IEEE 754 single-precision

Underflow and overflow can occur during computation.

- **Underflow.** If the exact result is non-zero and smaller than the smallest representable value, an *underflow* occurs, which may cause a loss of precision and create a significant computation error.
- **Overflow.** If the exact result is finite but exceeds the largest representable value, an *overflow* happens, which is another common cause of software failures.

When a processor detects an underflow or overflow, the hardware returns zero or the maximum number, respectively, and simultaneously generates an exception signal to provide an opportunity for the software to handle anomalies.

Overflow and underflow can cause strange and unexpected software failures that can be disastrous. On June 4, 1996, the Ariane 5 rocket veered off the designed course and destructed itself because an overflow occurred when assigning a floating-point number to an integer. That simple software failure caused an estimated loss of \$370 million.

At the hardware level, overflow and underflow can be detected by checking whether the exponent of a floating-point arithmetic operation is too large or too small. The processor can generate a particular hardware signal upon overflow or underflow.

However, it is difficult for programmers to write software to handle the anomaly because the signal can be raised at any point in the program. For example, in the following simple C statement, if the subtraction causes an underflow, most processors automatically return zero as the result of the subtraction. Consequently, a divide-by-zero anomaly takes place, even though the *if*-statement aims to prevent it.

```
if (a != b)
    c = 1/(a - b);    // Divide-by-zero can still occur.
```

Example 12-4. An if-statement fails to prevent divide-by-zero error.

12.2.4 Subnormal Numbers

IEEE 754 standard also defines a subnormal format for a class of floating-point numbers filling between 0 and the minimum positive number that the normalized format can represent. Upon an underflow in the normalized presentation, the data can be “deformatized” to the subnormal format to trade the numeric range for accuracy. Formerly the subnormal format was called *denormal* format.

A bit string is a subnormal format if all bits of the exponent are zero, but at least one bit of the fraction is non-zero. Specifically, for single-precision, the decimal value it represents can be calculated as follows:

$$(-1)^S \times \text{Fraction} \times 2^{-126}$$

Note the subnormal format differs from the normalized format introduced previously in two major aspects:

1. The subnormal allows a leading zero in front of the decimal point in the fraction, while the normalized format assumes the fraction always has a leading hidden 1 to the left of the decimal point. Thus, when calculating the decimal values, the normalized format uses $(1 + \text{Fraction})$ while the subnormal uses *Fraction* only.
2. All subnormal floating numbers have the same exponent, which is always 0.

The smallest positive number that the subnormal format in single-precision is

$$0,00000000,000000000000000000000001,$$

whose fraction value is 2^{-23} . Therefore, the smallest positive number is

$$2^{-23} \times 2^{-126} = 2^{-149} \approx 1.40 \times 10^{-45}$$

The largest positive number of single-precision subnormal format is

$$0,00000000,111111111111111111111111,$$

whose fraction value is $1 - 2^{-23}$.

Therefore, the largest positive subnormal number in the single-precision format is

$$\begin{aligned}(1 - 2^{-23}) \times 2^{-126} &= 2^{-126} - 2^{-149} \\ &\approx 1.175 \times 10^{-38}\end{aligned}$$

12.2.5 Tradeoff between Numeric Range and Resolution

Compared with the fixed-point system, the floating-point system with the same number of bits can represent a larger numeric range at the cost of degraded resolution. Specifically, all floating-point numbers are not distributed uniformly across the represented range.

If the total number of bits is given, the fixed-point format must play a tradeoff between the representation range and the representation resolution. It has either a large range but inferior resolution, or a narrow range but superior resolution.

Let us consider a hypothetical floating-point system that is like the IEEE 754 standard. This system has only five bits: the sign bit, an exponent with two bits and a fraction with two bits. The bias for the normalized representation is 1. Therefore, the corresponding decimal value is as follows:

$$(-1)^S \times (1 + \text{Fraction}) \times 2^{\text{Exponent}-1}$$

For example, if the binary is 10110, its corresponding floating-point number is

$$(-1)^1 \times (1 + 0.10)_2 \times 2^{2-1} = -3/4$$

If the exponent is zero but the fraction is not zero, then it is in subnormal format, and its decimal value is

$$(-1)^S \times \text{Fraction} \times 2^0 = (-1)^S \times \text{Fraction}$$

For example, if the binary is 10010, the decimal is then

$$(-1)^1 \times (0.10)_2 = -1/2$$

We consider a fixed-point system in the format of Q3.2, with three bits set aside for the integer portion and two bits used for the fractional portion. The integer portion uses two's complement. For a fair comparison, we also require the fixed-point system uses the same bit patterns to denote special values, including $\pm\infty$, QNaN, and SNaN.

Table 12-1 lists the values represented by all possible bit strings in both fixed-point Q3.2 and 5-bit floating-point systems.

Binary	Floating-point	Notation for floating-point	Fixed-point
00000	0.0	Reserved	0.0
00001	$\frac{1}{4}$	$(0.01)_2$	$\frac{1}{4}$
00010	$\frac{1}{2}$	$(0.10)_2$	$\frac{1}{2}$
00011	$\frac{3}{4}$	$(0.11)_2$	$\frac{3}{4}$
00100	1.0	$(1+0.00)_2 \times 2^{1-1}$	1.0
00101	$1\frac{1}{4}$	$(1+0.01)_2 \times 2^{1-1}$	$1\frac{1}{4}$
00110	$1\frac{1}{2}$	$(1+0.10)_2 \times 2^{1-1}$	$1\frac{1}{2}$
00111	$1\frac{3}{4}$	$(1+0.11)_2 \times 2^{1-1}$	$1\frac{3}{4}$
01000	2.0	$(1+0.00)_2 \times 2^{2-1}$	2.0
01001	$2\frac{1}{4}$	$(1+0.01)_2 \times 2^{2-1}$	$2\frac{1}{4}$
01010	3.0	$(1+0.10)_2 \times 2^{2-1}$	$2\frac{2}{2}$
01011	$3\frac{1}{4}$	$(1+0.11)_2 \times 2^{2-1}$	$2\frac{2}{2}$
01100	$+\infty$	Reserved	$+\infty$
01101	QNaN	Reserved	QNaN
01110	SNaN	Reserved	SNaN
01111	SNaN	Reserved	SNaN
10000	-0.0	Reserved	-0.0
10001	$-\frac{1}{4}$	$-(0.01)_2$	$-\frac{1}{4}$
10010	$-\frac{1}{2}$	$-(0.10)_2$	$-\frac{1}{2}$
10011	$-\frac{3}{4}$	$-(0.11)_2$	$-\frac{3}{4}$
10100	-1.0	$-(1+0.00)_2 \times 2^{1-1}$	-1.0
10101	$-1\frac{1}{4}$	$-(1+0.01)_2 \times 2^{1-1}$	-1 $\frac{1}{4}$
10110	$-1\frac{1}{2}$	$-(1+0.10)_2 \times 2^{1-1}$	-1 $\frac{1}{2}$
10111	$-1\frac{3}{4}$	$-(1+0.11)_2 \times 2^{1-1}$	-1 $\frac{3}{4}$
11000	-2.0	$-(1+0.00)_2 \times 2^{2-1}$	-2.0
11001	$-2\frac{1}{4}$	$-(1+0.01)_2 \times 2^{2-1}$	-2 $\frac{1}{4}$
11010	-3.0	$-(1+0.10)_2 \times 2^{2-1}$	-2 $\frac{2}{2}$
11011	$-3\frac{1}{4}$	$-(1+0.11)_2 \times 2^{2-1}$	-2 $\frac{2}{2}$
11100	$-\infty$	Reserved	$-\infty$
11101	QNaN	Reserved	QNaN
11110	SNaN	Reserved	SNaN
11111	SNaN	Reserved	SNaN

Table 12-1. Resolution and range comparison of floating-point and fixed-point numbers

Figure 12-13 plots the representable floating-point and fixed-point numbers listed in the above table on the same axis.

1. The fixed-point system has a smaller range of representable real numbers than the floating-point system.

2. The fixed-point system uniformly distributes representable numbers across the representable range, with same gaps between any two consecutive numbers. Thus, its resolution remains fixed. The resolution means the closeness of two consecutive numbers. However, the difference between two consecutive floating-point numbers becomes larger as the exponent increases. Accordingly, the resolution of the floating-point system degrades gradually as the absolute value increases. When we consider subnormal numbers, the resolution of the floating-point system becomes reasonable.

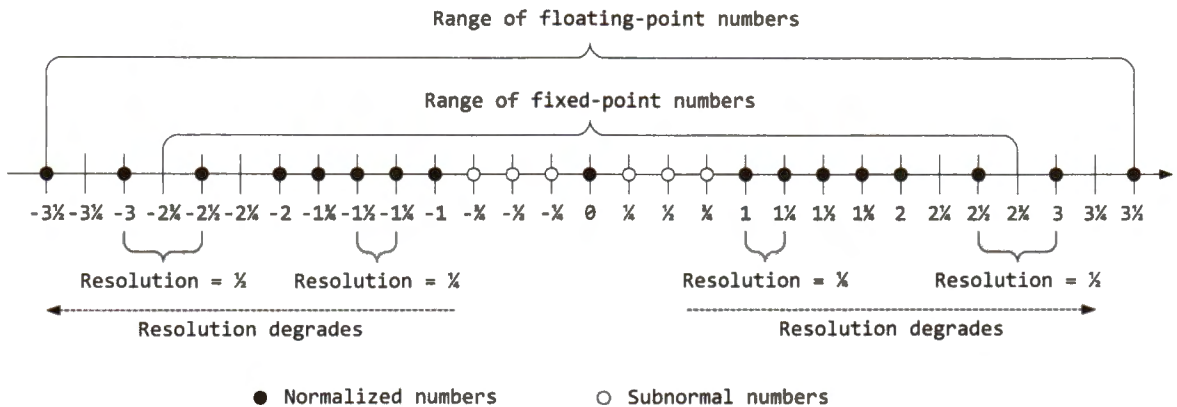


Figure 12-13. Comparing a floating-point and a fixed-point system with 5 bits. In the floating-point system, resolution degrades as the exponent increases. This is because the gap between two consecutive floating numbers increases. In this 5-bit example, overflow occurs if the absolute value is non-infinity but larger than $3\frac{1}{2}$. Underflow takes place if the absolute value is non-zero but smaller than $\frac{1}{4}$.

12.2.6 Rounding Rules

A finite number of binary digits cannot represent all real numbers without any error. Therefore, we must find an approximate of each un-representable real number. The process of finding a representable approximation is called *rounding*.

IEEE 754 defines four different rounding rules:

- Round to the nearest value,
- Round toward zero (truncation),
- Round toward $+\infty$ (round-up), and
- Round toward $-\infty$ (round-down).

When a number is rounded to the nearest and the number is exactly halfway between its two nearest numbers, we break the tie by choosing the one whose least significant bit in the fraction field is zero.

We use decimal numbers to illustrate the four rounding rules. To simplify the presentation, we assume only 5 digits are allowed after the decimal point, and Table 12-2 gives the rounded result of four rounding modes.

Rounding Rule	Data	Rounded Result
Nearest	+0.123456	+0.12346
	-0.123456	-0.12346
Truncate	+0.123456	+0.12345
	-0.123456	-0.12345
Rounding up	+0.123456	+0.12346
	-0.123456	-0.12345
Rounding down	+0.123456	+0.12345
	-0.123456	-0.12346

Table 12-2. Four different rounding rules

Rounding to the nearest is often the default rounding rule. The other three rounding rules are statistically biased.

- For example, when we repeatedly add a set of randomly generated numbers, the rounded sums are statistically smaller than the true sums if rounded down, and are statistically larger if rounded up.
- For a set of negative numbers, the sums are statistically larger than the exact sum if truncated.

Rounding to the nearest is unbiased statistically. Its error is symmetrical and has an equal chance of being positive and negative. It uses both truncate and round up.

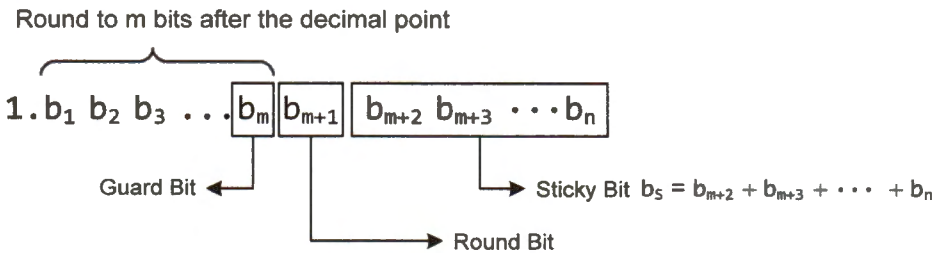


Figure 12-14. A binary number that is rounded to m bits after the point

Table 12-3 gives the algorithm of “rounding to nearest.” It keeps m bits to the right of the point, as shown in Figure 12-14. It checks three bits:

1. the guard bit b_m
2. the round bit b_{m+1}
3. the sticky bit b_s , which is the bitwise logical OR of all bits to the right of the round bit.

1. Calculate sticky bit $b_s = b_{m+2} + b_{m+3} + \dots + b_n$
2. If round bit $b_{m+1} = 0$, directly truncate and remove bits $b_{m+1}b_{m+2}b_{m+3}\dots b_n$
3. If round bit $b_{m+1} = 1$ and sticky bit $b_s = 1$, round up
4. If round bit $b_{m+1} = 1$ and sticky bit $b_s = 0$, round to even
 - If guard bit $b_m = 0$, directly truncate and remove $b_{m+1}b_{m+2}b_{m+3}\dots b_n$
 - If guard bit $b_m = 1$, round up

Table 12-3. Algorithm of rounding to nearest with ties rounding to even

As shown in Table 12-3, there are three different scenarios when a number is rounded to the nearest. In the last scenario, when the round bit b_{m+1} is 1 and the sticky bit b_s is 0, this is a tie. The value has an equal distance to two nearest neighbor numbers. The tie is broken by using the “round to even” rule, *i.e.*, choose the one that has 0 as its guard bit.

Table 12-4 gives a few examples to illustrate the above nearest rounding algorithm. Suppose we want to keep two binary digits after the binary point (*i.e.*, $m = 2$).

Binary Real Number	Round bit b_{m+1}	Sticky Bit b_s	Rounded Result	Round to Nearest Method
0.000001	0	Do not care	0.00	Truncate
0.000011	0	Do not care	0.00	Truncate
0.000101	0	Do not care	0.00	Truncate
0.000111	0	Do not care	0.00	Truncate
0.011000	1	0	0.10	Round to Even (Round up)
0.001000	1	0	0.00	Round to Even (Truncate)
0.001011	1	1	0.01	Round up
0.001101	1	1	0.01	Round up
0.001111	1	1	0.01	Round up

Table 12-4. Examples of rounding to nearest ($m = 2$)

Tiebreaker: Round to Even

We use 0.011000 as an example to illustrate the “round to even” rule for breaking the tie. It has two nearest values, 0.01 and 0.10. It is precisely halfway between these two nearest values. We use round-to-even to break the tie, *i.e.*, we select the closest value as the one in which the trailing bit is zero.

Specifically, the round bit is 1, and the sticky bit is 0. Thus, we should use the round to even method, *i.e.*, the step 4 of the rounding algorithm listed in Table 12-3. Because the guard bit is 1, we select the round-up method, and the result is 0.10.

Similarly, when 0.001000 is rounded to the nearest, it uses truncate to break the tie.

12.3 Software-based Floating-point Operations

A floating-point unit (FPU) is a coprocessor that carries out floating-point arithmetic operations. FPU is not available on Cortex-M3/M0, but it is optional on Cortex-M4/M7.

Let us compare the assembly implementation of floating point multiplication on Cortex-M processors with and without FPU. The following is a simple C function that calculates the area of a rectangle.

```
float area_of_rectangle(float length, float width) {
    float area;
    area = length * width;
    return area;
}
```

Example 12-5 shows the assembly implementation of the above C function.

- Without any FPU coprocessor, the compiler makes the program call the software floating point library to perform multiplication. The library uses integer-based instructions to implement floating-point multiplication.
- If FPU is available, multiplication can be carried out by simply calling the assembly instruction `VMUL.F32`. Note `s0` and `s1` are two registers of the FPU coprocessor, and they are different from `r0` and `r1` of the processor core.

Chapter 12.4 gives detailed information about FPU programming. This section focuses on software-based floating-point operations.

Software-based Multiplication	Hardware-based Multiplication
<pre>; Inputs: ; r0 = length ; r1 = width ; Return: ; r0 = area area_of_rectangle PROC PUSH {LR} ; area = length * width ; call software library BL __aeabi_fmuls POP {PC} ; return area in r0 ENDP</pre>	<pre>; Inputs: ; s0 = length ; s1 = width ; Return: ; s0 = area area_of_rectangle PROC ; area = length * width ; Call FPU instructions VMUL.F32 s0, s0, s1 BX lr ; return area in s0 ENDP</pre>

Example 12-5. Comparison of software- and hardware-based floating-point multiplication

12.3.1 Floating-point Addition

Assume two floating numbers have the same sign:

$$f_1 = (-1)^S \times (1 + F_1) \times 2^{E_1}$$

and

$$f_2 = (-1)^S \times (1 + F_2) \times 2^{E_2}$$

Without loss of generality, we also assume $E_1 \geq E_2$. Then mathematically, the addition is performed as follows:

$$\begin{aligned} f_1 + f_2 &= (-1)^S \times (1 + F_1) \times 2^{E_1} + (-1)^S \times (1 + F_2) \times 2^{E_2} \\ &= (-1)^S \times ((1 + F_1) + (1 + F_2) \times 2^{E_2 - E_1}) \times 2^{E_1} \\ &= (-1)^S \times \left((1 + F_1) + \frac{1 + F_2}{2^{E_1 - E_2}} \right) \times 2^{E_1} \end{aligned}$$

We can implement the division operation in the above equation by using shift operations.

When the sign of two floating-point numbers differs, the adjusted fraction parts are subtracted, instead of added.

As shown in Figure 12-15, the core procedure of adding two floating-point numbers involves the following steps:

- (1) shift the smaller fraction to match the larger one,
- (2) add or subtract the fraction based on its sign bits,
- (3) normalize the sum,
- (4) round the sum to appropriate bits, and
- (5) detect overflow and underflow.

The following illustrates the above algorithm by using a simple example. Suppose we are adding x and y , where

$$x = (1.1011011101)_2 \times 2^4$$

and

$$y = (1.1110101111)_2 \times 2^5$$

First, because the exponent of y is larger than x , we shift the fraction of x right by one bit to make the exponent match the larger one. Note we should not shift left the fraction of the operand with a larger exponent to match the lower exponent. The reason is that we may create a significant computational error if a larger operand causes an overflow when it is shifted.

$$x = (0.110110111101)_2 \times 2^5$$

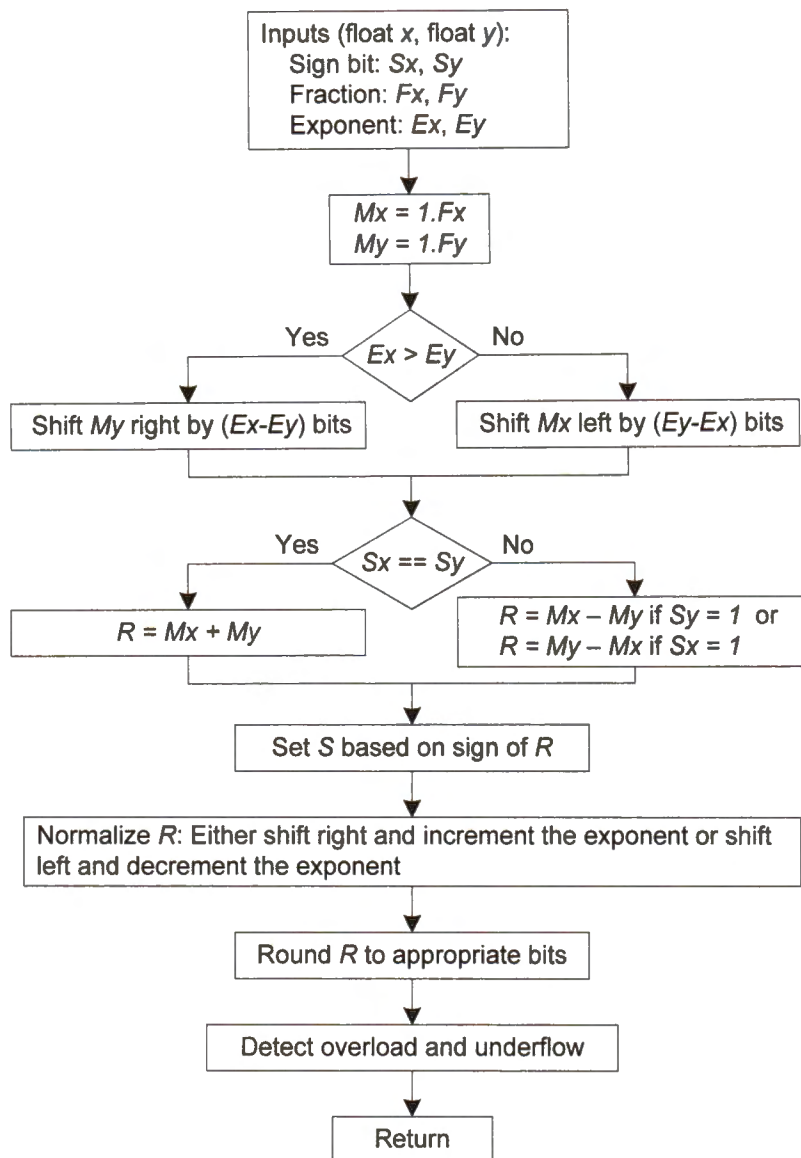


Figure 12-15. Flowchart of adding two floating numbers

Then we perform a simple addition. We do not need to worry about the alignment of the decimal points because both fraction bit strings have the same length.

$$\begin{array}{r}
 x = \quad 0 \quad . \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 1 \\
 y = \quad 1 \quad . \quad 1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad 0 \\
 \hline
 x + y = 1 \quad 0 \quad . \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1
 \end{array}$$

Thus, we get the sum:

$$x + y = (10.11000111011)_2 \times 2^5$$

The next step is to normalize the sum. In this case, we should shift the sum right by one bit because the integer part of the fraction of the sum has two bits. If the fraction of the sum has many leading zeros, including the integer part, then we should shift the fraction of sum left and reduce the exponent accordingly. The following is the sum after normalization.

$$x + y = (1.011000111011)_2 \times 2^6$$

During the normalization, the hardware can detect overflow or underflow by checking whether the result falls out of the representable data range.

While the implementation is complex because it needs to consider special inputs, such as $\pm\infty$ and NaN, the following shows a simplified implementation that ignore special inputs and does not detect overflow and underflow.

```
; Adding two floating-point numbers
; Input: r0 = first operand, r1 = second operand
; Output: r0 = sum
;
; For each register,
;   bits[31] = sign (1 bit)
;   bits[30-23] = exponent (8 bits)
;   bits[22:0] = fraction (23 bits)

fadd PROC
; If inputs have different signs, flip the sign bit of the second operand
; and call the subtraction function.
TEQ    r0, r1                ; If r0 and r1 have the same sign, then N = 0
                                ; Otherwise N = 1
EORMI  r1, r1, #0x80000000   ; If N = 1, flip sign bit of second operand
BMI    fsub                  ; call subtraction, return r0 - (-r1)

; r0 and r1 are guaranteed to have the same sign
; If r0 < r1 (unsigned comparison), swap them
SUBS   r2, r0, r1            ; r2 = r0 - r1
SUBSCC r0, r0, r2            ; if r0 < r1, swap r0 and r1
ADDSCC r1, r1, r2            ; Guarantee r0 > r1

; Find the difference of exponents
; Now it has guaranteed that r0 > r1 (unsigned comparison)
; r3[8:0] = [sign:exponent] of r0 - [sign:exponent] of r1
LSR    r2, r0, #23           ; r2[8:0] = [sign:exponent] of r0
SUB     r3, r2, r1, LSR #23   ; r3 = difference of exponents

; Shift right the fraction of the smaller operand (r1) to make
; their exponents match and then add the fractions
MOV     r12, #0x80000000      ; Set the leading hidden 1
```

```

ORR    r0, r12, r0, LSL #8 ; r0 = 1:fraction:0000,0000
ORR    r1, r12, r1, LSL #8 ; r1 = 1:fraction:0000,0000
LSR    r12, r1, r3          ; r12 = r1 >> difference between exponents
ADDS   r12, r0, r12         ; r12 = sum of both fractions
BCS    fraction_too_large   ; If fraction sum is 11.xxxx or 10.xxxx, skip

; Pack final result into r0
; The actual value of the fraction sum is 1.xxxxx
LSRS   r0, r12, #8          ; r0[22:0] = first 23 bits of fraction of sum
                        ; Guard bit is shifted into carry
ADC    r0, r0, r2, LSL #23 ; r0[31:23] = [sign:exponent] of sum,
                        ; also add guard bit into r0
BXCC   lr                  ; if guard bit is 0, return truncated result

fraction_too_large
; The actual value of the fraction sum is 11.xxxxx or 10.xxxxx
RRX    r0, r12, r1          ; Shift carry back into r0

; If guard bit is 1, perform rounding to even or round up
; and pack final result into r0
RSB    r3, r3, #32          ; r3 = 32 - r3, r3 = diff of exponents
LSLS   r1, r1, r3          ; r1 = all sticky bits
BICEQ  r0, r0, #1          ; Clear the round bit if all sticky bits
                        ; are zero (round to even)
LSRS   r0, r0, #8          ; r0[22:0] = first 23 bits of fraction of sum
                        ; Guard bit is shifted into carry
ADC    r0, r0, r2, LSL #23 ; r0[31:23] = [sign:exponent] of sum
                        ; also add guard bit into r0
BX     lr                  ; return
ENDP

```

Example 12-6. Simplified implementation of adding two floating-point numbers

The sum of adding two aligned fraction parts (including their leading hidden 1 bit) can have three possible results, 1.xxxxx, 10.xxxxx, or 11.xxxxx. When the integer part has two bits (10.xxxxx or 11.xxxxx), we need to shift the leading bit back to the fraction of the sum by running the instruction “RRX r0, r12, #1”.

Furthermore, when the statement “ADC r0,r0,r2,LSL #23” packs the result into the single-precision format, the leading bit of the fraction sum is added to the exponent part because the leading bit is removed from the fraction sum before the packing operation. Therefore, if the fraction sum is 10.xxxxx or 11.xxxxx, then the “ADC” instruction automatically increases the exponent of the result by 1.

As the standard suggests, the default rounding should be the nearest rounding. When packing the results, “ADC” instead of “ADD” is used. The carry bit is added to the fractional part of the result to achieve the nearest rounding. In fact, the carry bit holds the guard

bit. If the guard bit is 0, extra bits are truncated by the “LSRS r0, r0, #8” instruction. If all sticky bits are 0, then the round bit (the least significant bit of the fraction sum) is set to zero. If the sticky bits are not all zero, the guard bit is 1 and is added to the result.

12.3.2 Floating-point Multiplication

Suppose we multiply the following two floating-point numbers:

$$f_1 = (-1)^{S_1} \times (1 + F_1) \times 2^{E_1}$$

$$f_2 = (-1)^{S_2} \times (1 + F_2) \times 2^{E_2}$$

Mathematically, the product of two floating-point numbers

$$\begin{aligned} f_1 \times f_2 &= ((-1)^{S_1} \times (1 + F_1) \times 2^{E_1}) \times ((-1)^{S_2} \times (1 + F_2) \times 2^{E_2}) \\ &= (-1)^{S_1+S_2} \times (1 + F_1) \times (1 + F_2) \times 2^{E_1+E_2} \end{aligned}$$

As the above equation indicates, the multiplication involves the following four steps: (1) identify the sign of the product, (2) add the exponents together, (3) multiply the fractions, including the leading hidden one, and (4) normalize the result to the standard format.

The following gives a simplified implementation of multiplying two single-precision floating point numbers. The program does not handle special cases, such as overflow, underflow, and input of infinity and NaN.

The trick is to manage the multiplication of two fractions. Two fractions include their hidden leading 1. When a program multiplies two fractions of the same length, *i.e.*,

$$1.xx...x \times 1.yy...y$$

where x and y are either 1 or 0, their product must be truncated to the same length. Suppose each fraction has n bits, their product has either $2n$ or $2n - 1$ bits because the leading bit of both fractions is 1.

We can use $n = 2$ as a simple example. In binary, we have three different scenarios:

$$1.1 \times 1.1 = 10.11$$

$$1.1 \times 1.0 = 01.10$$

$$1.0 \times 1.0 = 01.10$$

In other words, the most significant two bits of the product of $1.xx...x$ and $1.yy...y$ can only be either 10 or 01.

- If the most significant bits are 01, we need to remove the leading zero bit.
- If the most significant bits are 10, we need to renormalize the result by shifting the fraction product right by one bit and then increasing the exponent by 1.


```

; Function of multiplying two single-precision floating-point numbers
; Input: r0 = first operand, r1 = second operand
; Output: r0 = product
; For each register,
;     bits[31] = sign (1 bit)
;     bits[30-23] = exponent (8 bits)
;     bits[22:0] = fraction (23 bits)

fmul PROC
; Add two exponents
MOV     r12, #0x000000FF          ; mask
ANDS    r2, r12, r0, LSR #23      ; r2[7:0] = exponent of r0
ANDS    r3, r12, r1, LSR #23      ; r3[7:0] = exponent of r1
ADD     r2, r2, r3                ; r2[7:0] = sum of both exponents

; If operands have different signs, set the product as negative
TEQ     r0, r1                    ; check whether r0 and r1 have different signs
ORRMI   r2, r2, #0x100            ; i.e., check r2[8] = 1

; Multiply fractions
MOV     r12, #0x80000000          ; Mask for leading hidden 1
ORR     r0, r12, r0, LSL #8        ; r0 = 1:Fraction:00000000 (binary)
ORR     r1, r12, r1, LSL #8        ; r1 = 1:Fraction:00000000 (binary)
UMULL   r1, r3, r0, r1            ; r1:r3 = 64-bit product of both fractions

; Perform rounding up and discard lower 32 bits of the fraction product
CMP     r1, #0                    ; Are tailing 32 bits of the product
                                           ; of two fractions are zero?
ORRNE   r3, r3, #1                ; if r1 ~= 0, round up, set r3[0] = 1

; Remove Leading zero bit of fraction product
; Note there is at most one leading zero bit in the product
LSLS    r3, r3, #1                ; r3 << 1, shift out leading bit
RRXCS   r3, r3, #1                ; If the leading bit was 1, recover it

; Pack final result into r0
LSRS    r12, r3, #8               ; r12 = 0000,0000:24-bit fraction (binary)
                                           ; Guard bit is shifted into carry
ADC     r0, r12, r2, LSL #23       ; Pack result into r0
                                           ; r2[8:0] = [sign:exponent] of product
                                           ; Guard bit (carry bit) is added to fraction
BXCC    lr                        ; If the guard bit is 0, return
LSL     r12, r3, #24               ; r12 = [guard bit : all sticky bits]
CMP     r12, #0x80000000           ; if guard bit is zero and sticky bits are
                                           ; zero, clear the last bit of fraction
BICEQ   r0, r0, #1                ; round to even by setting r0[0] = 0;
BX      lr                        ; return

ENDP

```

Example 12-7. Simplified implementation of multiplying two floating-point numbers

12.4 Hardware-based Floating-point Operations

Cortex-M4 and Cortex-M7 can optionally have a floating-point Unit (FPU) coprocessor. An FPU coprocessor has its own data and control registers, and it supports single-precision arithmetic, accessing memory data, integer and precision conversion, and some double-precision arithmetic. This section introduces how to program FPU and handle exceptions in assembly.

12.4.1 FPU Registers

The FPU consists of 32 single-word general-purpose registers ($S0, S2, \dots, S31$) and four special-purpose registers (CPACR, FPCCR, FPCAR, and FPSCR), as shown in Figure 12-16.

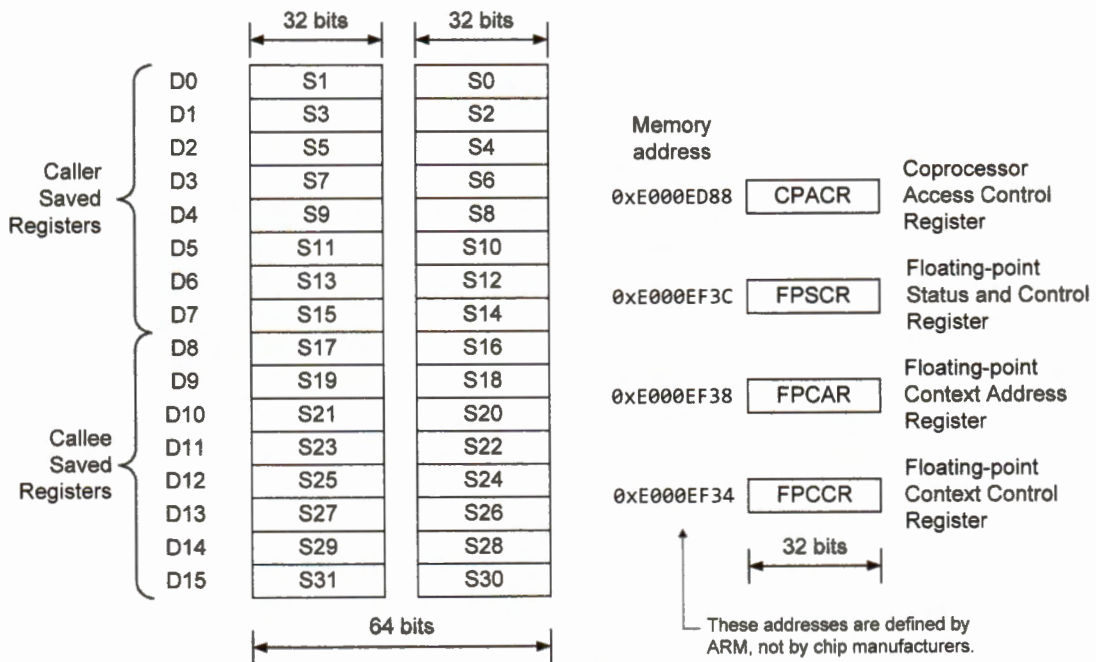


Figure 12-16. Floating-point registers (32 general-purpose 32-bit registers and 4 special registers). The processor remaps two 32-bit general-purpose registers to a 64-bit register. For example, $S1$ and $S0$ are remapped to $D0$. $D0$ uses the same hardware resources as $S1$ and $S0$.

The callee subroutine must preserve registers $D8$ - $D15$ or $S16$ - $S31$ if it writes to them.

12.4.1.1 Floating-point General-purpose Registers ($S0$ - $S31$, or $D0$ - $D15$)

Each general-purpose floating-point register ($S0, S2, \dots, S31$) can hold one single-precision floating number. The prefix "S" stands for single-precision.

These 32 general-purpose floating-point registers can be mapped into 16 double-word registers ($D0, D1, \dots, D15$) for holding double-precision floating point numbers. The prefix

“D” stands for double-precision. For example, a double-precision register D0 consists of single-precision registers S1 and S0, with S1 holding the most significant word of D0, and S0 holding the least significant word of D0.

When a subroutine takes single- or double-precision floating point numbers as input arguments, registers S0-S15 or D0-D7 hold these input arguments.

- If the subroutine takes only single-precision numbers as input arguments, up to 16 arguments can be passed via registers. These arguments come in via registers S0-S15 in a sequential order. If more than 16 arguments are passed, the caller must push all additional arguments onto the stack.
- If only double-precision numbers are passed, up to 8 arguments can be passed via registers D1-D7. Extra arguments, if needed, should be passed via the stack.
- If a mixed of single-precision and double-precision numbers are passed, each number is assigned in turn to the next free register of the corresponding type.

Figure 12-17 shows an example that passes a mixed of float and double variables to a subroutine.

```
double fun(double a1, float a2, double a3, float a4, float a5, double a6, float a7, double a8)
```

	64 bits															
Double-precision View	D0		D1		D2		D3		D4		D5		D6		D7	
Single-precision View	S0	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15
Argument View	a1		a2	a4	a3		a5	a7	a6		a8					

Figure 12-17. Passing arguments to a subroutine via floating-point registers

If a subroutine returns a floating-point number, the return value is saved in register S0 for single-precision or D0 for double-precision. For other data types, such integers, the return value is in register r0-r4 (in Chapter 8).

When a subroutine takes as an input argument a pointer that points a single- or double-precision floating-point value, the pointer is not a floating-point value but a memory address, and thus it is passed via an integer register, instead of a floating-point register.

Example: passing float/double array pointers

```
float * fun(float * array1, double * array2, int array_size)
```

The variable array1 and array2 are array pointers, and they are passed via register r0 and r1, respectively. The array_size is passed in register r2. Additionally, this function returns a float pointer, which is saved in register r0.

12.4.1.2 Coprocessor Access Control Register (CPACR)

Cortex-M has two coprocessors: CP10 and CP11. They are vector floating point (VFP) coprocessors and carry out floating point arithmetic operations.

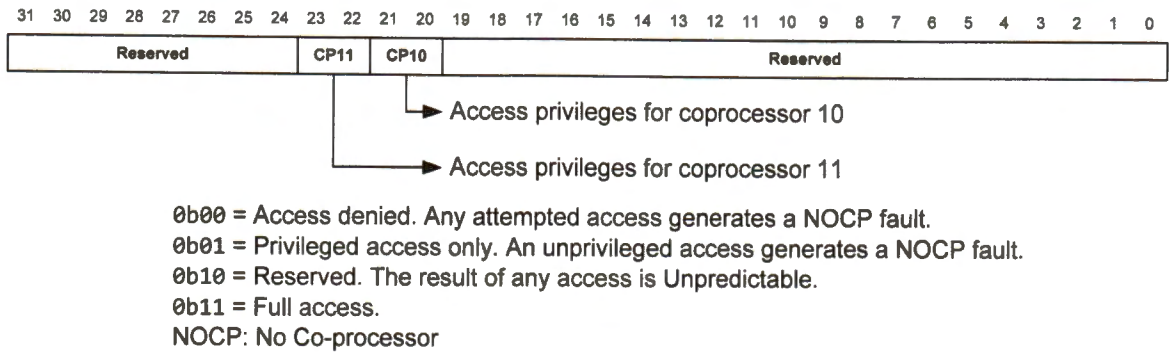


Figure 12-18. Coprocessor Access Control Register (CPACR)

FPU is disabled by default. To use FPU, software must enable CP10 and CP11. The program should use the data synchronization barrier (DSB) to ensure the completion of all memory accesses. The instruction synchronization barrier (ISB) should also be used to flush the processor pipeline and ensure that all instructions are fetched from cache or memory again.

C Program	Assembly Program
<pre> void FPU_Enable(void) { // Enable CP10 and CP11 SCB->CPACR = (0xF << 20); // Ensure the completion of // memory accesses __DSB(); // Flush the processor pipeline // fetch buffers __ISB(); return; } </pre>	<pre> FPU_Enable PROC EXPORT FPU_Enable ; Load SCB base address (0xE000ED00) LDR r0, = SCB_BASE ; Read from SCB->CPACR LDR r1, [r0, #SCB_CPACR] ; Enable full access to CP10 and CP11 ORR r1, r1, #(0xF << 20) ; Write to SCB->CPACR STR r1, [r0, #SCB_CPACR]; DSB ; Data Synchronization Barrier ISB ; Instruction Syn. Barrier BX LR ; Return ENDP </pre>

Example 12-8. Enabling VFP coprocessors

Similarly, to disable FPU, software must clear the enable bits of coprocessors 10 and 11 used by FPU: "SCB->CPACR &= ~(0xF << 20);".

Additionally, the processor must be in the privileged mode to enable or disable FPU. FPU must be enabled before any floating-point instruction is executed; otherwise, the processor generates a NOCP (No Coprocessor) usage fault. In most template projects provided in ARM Keil, we can enable FPU in the start-up assembly file (such as *startup_stm32f407xx.s*) before the instruction “BL __main”. Chapter 23.1 introduces the processor modes.

12.4.1.3 Floating-point Status and Control Register (FPSCR)

The FPSCR register stores the FPU configuration, the condition flags and the exception flags. We only use VMRS to read FPSCR and VMSR to write to it.

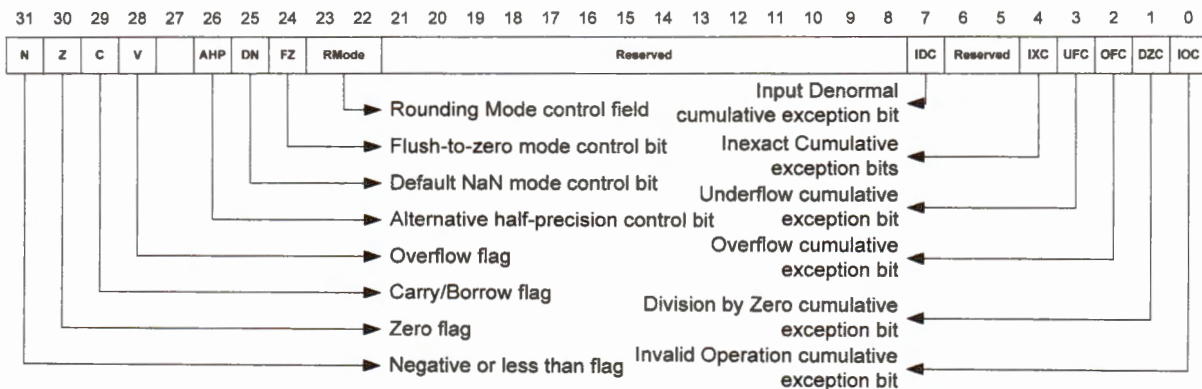


Figure 12-19. Floating-Point Status and Control Register (FPSCR)

N, Z, C, V Flags. The FPSCR register has the negative (N), zero (Z), carry (C), and overflow (V) flag bits. These bits are set by the single-precision comparison instructions VCMP.F32 and VCMPE.F32 (to be introduced later in this chapter).

The processor cannot directly use these flags to perform the conditional execution. Software program must use the VMRS instruction to transfer these flags to the processor's APSR for conditional execution, as shown below.

```
VMRS r4, FPSCR           ; Copy FPSCR to register r4
VMSR APSR_NZCV, FPSCR   ; Copy N, Z, C, V flags from FPSCR to APSR
```

APSR_nzcv includes a bit specifier to specify the APSR bits to be updated. The following gives a few examples of the bit specifier.

- APSR_NZCV is to access the N, Z, C, V bits in APSR.
- APSR_G is to access the GE[3:0] bits in APSR.
- APSR_NZCVQ is to access the N, Z, C, V, Q bits in APSR.
- APSR_NZCVQG is to access the N, Z, C, V, Q, GE[3:0] bits in APSR.

Rounding Modes. FPU carries out floating-point arithmetic with extra precision, and then rounds the result to fit into the destination precision. This internal extra precision ensures arithmetic results are as precise as the source operands. FPU supports four rounding modes, determined by two RMode control bits in FPSCR (*i.e.*, FPSCR[23:22]), as described below.

- 00: Round to nearest mode (default)
- 01: Round towards $+\infty$ mode
- 10: Round towards $-\infty$ mode
- 11: Round towards 0 mode

```
FPU->FPDSCR &= ~FPU_FPDSCR_RMode_Msk; // Clear rounding mode bits
FPU->FPDSCR |= 2 << 22; // Select round toward positive infinity
```

Example 12-9. Selecting round to $+\infty$

The ARM FPU architecture provides three special modes, including the flushing-to-zero mode, the default NaN mode, and the alternative half-precision mode. These three modes are not compatible with the IEEE 754 standard.

- The *flush-to-zero mode* can be enabled by setting the FPSCR[24] bit. If this mode is enabled, all operands in the subnormal format are flushed to zero, except for instructions VABS, VNEG, and VMOV. Specifically, the input or the result of a single-precision arithmetic operation is treated as zero if its absolute value is larger than zero but smaller than 2^{-126} . If this mode is enabled, the FPU computation speed is improved at the expense of some accuracy.
- The *default NaN mode* can be enabled by setting the FPSCR[25] bit. NaN stands for not a number. As shown in Figure 12-11, there are many possible NaNs. In the IEEE 754 standard, two NaNs are different if they differ in any bit. IEEE 754 requires that any operation, which takes one or more quiet NaN operands, should return the input NaN as its result. If the default NaN mode is disable, FPU strictly follows IEEE 754 standard. However, if the default NaN mode is enabled, the result of any operation involving one or more NaNs is always the same pre-defined quiet NaN, which is $0x7FC00000$.
- The *alternative half-precision mode* can be enabled by setting the FPSCR[26] bit. It selects either IEEE 754 half-precision floating point representation or ARM Cortex special 16-bit mode. The ARM alternative offers a larger range than IEEE 754 but does not support infinity and NaN.

Exceptions arise during abnormal events. Floating-point numbers are finite-precision approximation of real numbers. A floating-point operation may have range violation (overflow or underflow), rounding errors (inexact), invalid operation, or divide-by-zero. Cortex-M supports the following six exceptions. Once these exception flags are set, hardware does not automatically reset them.

- The *underflow exception* (UFC) and the overflow exception (OFC) are set respectively if the result has an absolute value greater than the largest floating-point number, or it is non-zero and smaller than the smallest non-zero floating-point number.
- The *inexact exception* flag (IXC) is set if the result of a single-precision float operation does not equal to the value that would be obtained if the operation were performed with unbounded precision and exponent range. This happens if the result lies between two floating-point numbers, leading to a rounding error.
- The *invalid operation* exception flag (IOC) is set if the result has no mathematical value or cannot be represented, such as $0 \times (\pm\infty)$, $\infty + (-\infty)$, $(\pm\infty)/(\pm\infty)$, and $\text{sqrt}(-1.0)$. The default output is either SNaN or QNaN.
- The *divide-by-zero* (DZC) exception flag is set if the divisor of a divide operation is zero, but its dividend is not zero.
- If a value is flushed to zero, the *input denormal cumulative* (IDC) exception flag bit of FPSCR is set.

12.4.1.4 Floating-point Context Address Register (FPCAR)

As introduced previously in Chapter 11, the processor performs automatic stacking when an interrupt/exception occurs. Without FPU support, the processor automatically pushes eight registers, including $r0$ - $r3$, $r12$, LR, PC, and xPSR, onto the stack.

If FPU auto-stacking has been enabled (see the next section), the processor automatically pushes 17 additional registers, including $s0$ - $s15$ and FPSCR onto the stack when interrupt takes place. The processor stores the stack location holding the value of register $S0$ to the floating-point context address register (FPCAR), as shown in Figure 12-20.

Additionally, the processor uses *lazy stacking* to reduce the interrupt latency. The lazy stacking allows the processor to skip the stacking of FPU registers, if not required, *i.e.*,

- if the interrupt handler does not use FPU, or
- if the interrupted program has not used FPU.

The processor reserves space in the stack for FPU registers. However, it only pushes FPU registers if FPU has been used and the interrupt handler will also FPU. Register FPCAR points to the unpopulated stack location reserved for FPU register S0 if stacking is deferred.

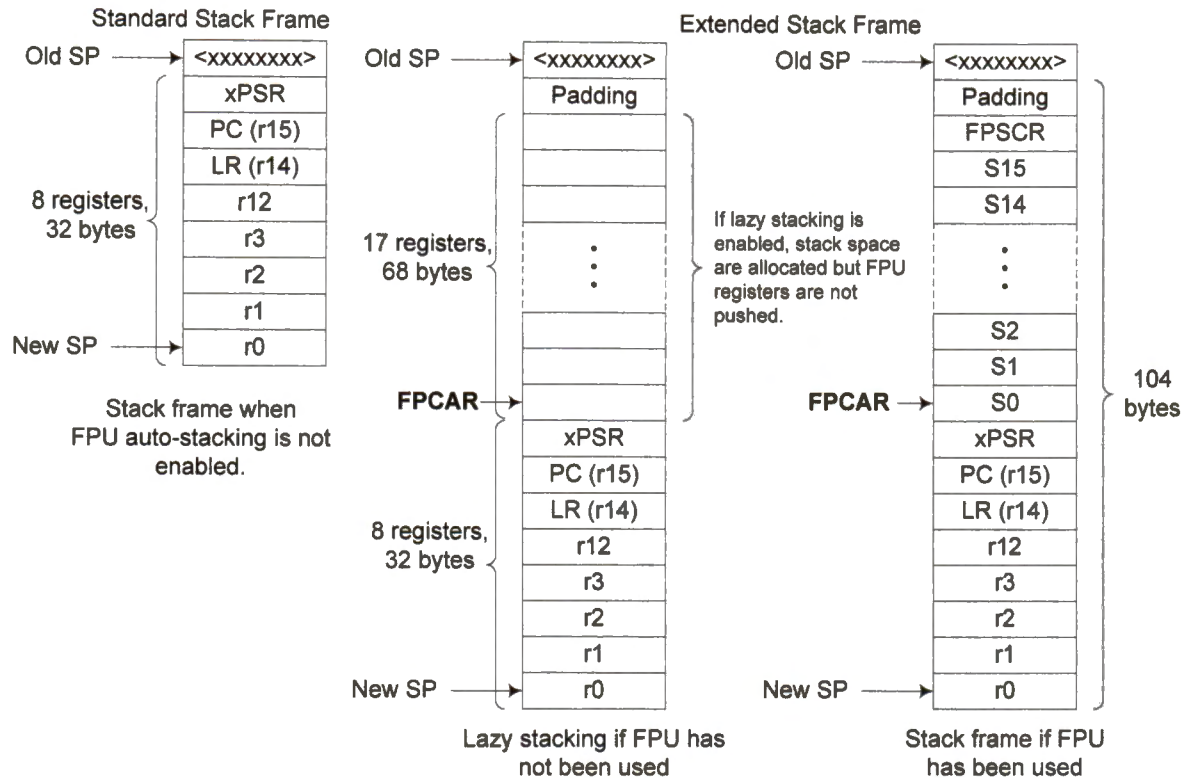


Figure 12-20. Lazy stacking on interrupt entry defers pushing FPU registers if FPU has not been used or the interrupt handler does not use FPU. FPCAR points to the memory location reserved for register S0. A 4-byte padding is added to make SP aligned to double words.

When the processor services an interrupt, it decides at runtime whether these FPU registers should be stored in the reserved stack space. The decision is made by checking the floating-point context active (FPCA) bit flag in the CONTROL register, which is a special register on Cortex-M processors.

- When the processor runs a floating-point instruction, it sets the FPCA bit flag in the CONTROL register automatically. The FPCA bit is automatically cleared when a new context, such as an interrupt service routine, is started.
- When the FPCA bit in the CONTROL register is 1 and the interrupt handler first uses any FPU instruction, the processor automatically pushes 17 additional registers onto stack and stores in the FPCAR register the memory address where the value of register s0 is in the stack.

The following example shows how to retrieve the previous FPSCR value from the stack, which is for the floating-point operations before the interrupt take places. The interrupt handler performs a dummy read to force the deferred FPU stacking in case that lazy stacking has been used.

```
void Example_IRQHandler(void) {
    uint32_t status, *fpscr;

    // Get the address where FPSCR is stored in the stack
    *fpscr = (uint32_t *) (FPU->FPCAR + 64); // Full descending stack

    // Dummy access to populate the stack in case that lazy stack is active
    (void) __get_FPSCR(); // dummy read to FPSCR to force state preservation

    // Read FPSCR from stack
    status = *fpscr;

    // Check exception flag bits
    ...
}
```

Example 12-10. Retrieve FPSCR of previous FPU operations from the stack

12.4.1.5 Floating-point Context Control Register (FPCCR)

The Floating-point Context Control Register (FPCCR) controls the behavior of context saving and restoring during an interrupt. There are three different settings.

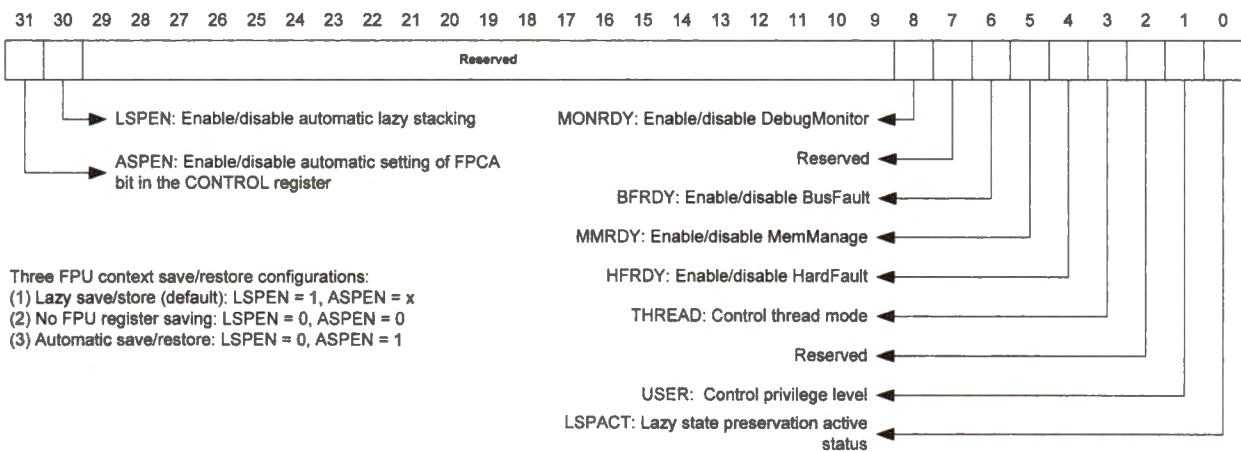


Figure 12-21. Floating-point Context Control Register (FPCCR)

- By default, the LSPEN (lazy state preservation enable) bit is 1, and it allows *lazy stacking* to reduce the exception or interrupt latency. When space has been allocated on the stack for FPU registers but pushing them onto the stack has been deferred, the lazy state preservation active status (LSPACT) flag is set.

- If both LSPEN and ASPEN (always state preservation enable) are 0, no floating-point registers are saved in the stack by the hardware when an interrupt occurs.
- If LSPEN is 0 and ASPEN is 1, floating-point registers are always automatically saved onto the stack by the hardware upon an interrupt.

Example 12-11 gives C functions that modify the stacking settings.

```
void FPU_enableAutoStacking_disableLazyStacking(void) {
    // Disable lazy stacking
    FPU->FPCCR &= ~FPU_FPCCR_LSPEN_Msk;
    // Enable automatic stacking
    FPU->FPCCR |= FPU_FPCCR_ASPEN_Msk;
}

void FPU_enableStacking(void) {
    // Enable automatic and lazy stacking
    FPU->FPCCR |= FPU_FPCCR_ASPEN_Msk | FPU_FPCCR_LSPEN_Msk;
}

void FPU_disableStacking(void) {
    // Disable automatic and lazy stacking
    FPU->FPCCR &= ~(FPU_FPCCR_ASPEN_Msk | FPU_FPCCR_LSPEN_Msk);
}
```

Example 12-11. Enable and disable FPU automatic and lazy stacking

The FPCCR register also controls whether the processor can raise exceptions when it allocates the floating-point stack frame. These exceptions include the debug monitor exception, the bus fault, the memory management exception, and the hard fault.

When the floating-point stack frame is allocated, FPCCR also provides two flag bits, including USER and THREAD, to indicate whether the processor is in the user mode and the privileged mode, respectively.

If lazy stacking is enabled, the *lazy state preservation active* (LSPACT) bit in the FPCCR register indicates whether actual stacking of FPU registers has taken place. The processor sets LSPACT to 1 if stack space has been allocated but FPU stacking has been deferred on the entry of the interrupt handler. If LSPACT is 1, the processor stops the pipeline and starts to perform the deferred FPU stacking when the current interrupt handler runs a floating-point instruction. After FPU stacking completes, the processor clears the LSPACT flag to zero. Then, the processor continues the execution of the current interrupt handler.

12.4.2 Load and Store Floating-point Numbers

The following instructions access floating-point numbers stored in memory.

VLDR.F32 Sd, [Rn]	Load one single-precision float
VLDR.F64 Dd, [Rn]	Load one double-precision float
VSTR.32 Sd, [Rn]	Store one float registers
VSTR.64 Dd, [Rn]	Store one double register
VLDM.32 Rn{!}, list	Load multiple single-precision floats
VLDM.64 Rn{!}, list	Load multiple double-precision floats
VSTM.32 Rn{!}, list	Store multiple float registers
VSTM.64 Rn{!}, list	Store multiple double registers
VPOP.32 list	Pop float registers from stack
VPOP.64 list	Pop double registers from stack
VPUSH.32 list	Push float registers to stack
VPUSH.64 list	Push double registers to stack

The memory access instructions VLDR and VSTR support three addressing modes:

```
VLDR.F32 s0, [r0, #4] ; Pre-index, s0 = mem[r0 + 4]
VLDR.F32 s0, [r0], #4 ; Post-index, s0 = mem[r0], r0 += 4
VLDR.F32 s0, [r0, #4]! ; Pre-index with update, s0 = mem[r0 + 4], r0 += 4
```

Multiple floating-point numbers can be loaded or stored in one instruction:

```
VLDM.64 r0, {d0-d7} ; Load 8 double precision numbers
VSTM.64 r0, {d0-d7} ; Store 8 double precision numbers
```

Like LDM and STM introduced in Chapter 5.5, VSTM and VLDM can also have the suffix IA (Increment After) and DB (Decrement After), such as,

```
VSTMIA.64 r0, {d0-d7}
VSTMDB.64 r0, {d0-d7}
```

The push and pop operations are useful to save and restore registers via the stack.

```
VPUSH.32 {s16-s31} ; Save floating point registers
VPOP.32 {s16-s31} ; Restore floating point registers
```

For the convenience of the programmer, VLDR.F32 and VLDR.F64 can be used as a pseudo-instruction to load a constant into a floating-point system register.

```
VLDR.F32 s0, =2.71828
VLDR.F64 d2, =3.14159
```

The compiler translates the constant into IEEE 754 format, stores it in the instruction memory, and then uses PC-relative addressing to load the constant from memory. The compiler determines the offset relative to PC. When the processor calculates the target address, bit PC[1] is ignored to ensure that the target address is word-aligned.

```
VLDR.F32 s0, [pc, #0xA8] ; Load a float from PC + 4 + 0xA8
VLDR.F64 d2, [pc, #0xC4] ; Load a double from PC + 4 + 0xC4
```

12.4.3 Copy Floating-point Numbers

The following table lists instructions that copy floating-point numbers to a register.

VMOV.F32 Sd, #imm	Move an immediate float to float-register
VMOV.F32 Sd, Sm	Copy from single-precision register to single-precision register
VMOV.F64 Dd, Dm	Copy from double-precision register to double-precision register
VMOV Sn, Rt	Copy ARM core register to float register
VMOV Sm1, Sm2, Rt1, Rt2	Copy 2 ARM core registers to 2 float registers
VMOV Dd[x], Rt	Copy ARM core register to a half of a double-precision floating point register, where x is 0 or 1.
VMOV Rt, Dn[x]	Copy a half of a double-precision floating point register to ARM core register, where x is 0 or 1.

The VMOV instructions can be transfer content between ARM core registers and FPU coprocessor registers. For example,

```
VMOV s0, s1, r0, r1      ; s0 = r0, s1 = r1
VMOV r1, d0[1]           ; r1 = d0[1], i.e., r1 = s1
```

Note we can also use VMOV to initialize a single-precision register with a constant.

```
VMOV.F32 s1, #3.25       ; s1 = 3.25
```

12.4.4 Copy and Set the Status and Control Register

The FPSCR register holds the N, Z, C, and V flags, exception flags, and FPU configuration. VMRS and VMSR are special instructions to transfer contents between FPSCR and a general-purpose register.

VMRS Rt, FPSCR	Copy FPSCR to ARM core register or APSR
VMSR FPSCR, Rt	Copy to FPSCR from ARM Core register

The following shows example codes to set the rounding mode to round toward zero.

```
VMRS r0, FPSCR           ; Read current FPSCR
ORR  r0, r0, #(3<<22)    ; Set bits 22 and 23 to 1
VMSR FPSCR, r0           ; Save FPSCR
```

Note that conditional execution of floating-point arithmetic operations, such as VMOVL and VADDEQ.F32, evaluate the N, Z, C, and V flags of APSR, not FPSCR.

The following instruction copies the N, Z, C, and V flags from FPSCR to APSR.

```
VMRS APSR_nzcv, FPSCR    ; Copy NZCV flags from FPSCR to APSR
```

12.4.5 Single-precision Arithmetic Operations

FPU in Cortex processors supports a limited number of single-precision floating point arithmetic operations, as listed below.

VADD.F32 {Sd,} Sn, Sm	Add floating points, $Sd = Sn + Sm$
VSUB.F32 {Sd,} Sn, Sm	Subtract float, $Sd = Sn - Sm$
VDIV.F32 {Sd,} Sn, Sm	Divide single-precision floats, $Sd = Sn/Sm$
VMUL.F32 {Sd,} Sn, Sm	Multiply float, $Sd = Sn * Sm$
VNMUL.F32 {Sd,} Sn, Sm	Negate and multiply float, $Sd = -1 * Sn * Sm$
VNEG.F32 Sd, Sm	Negate float, $Sd = -1 * Sm$
VABS.F32 Sd, Sm	Absolute value of floats, $Sd = Sm $
VSQRT.F32 Sd, Sm	Square-root of float

The following program shows a subroutine in C and assembly that calculates the area of a circle. The subroutine takes the input argument radius via the single-precision floating point register `s0`. The calculated area is also returned in register `s0`.

C Function	Assembly Function
<pre>float area_of_circle(float radius){ float PI = 3.14; float area; area = PI * radius * radius; return area; }</pre>	<pre>area_of_circle PROC EXPORT area_of_circle VLDR.F32 s1,=3.14 ; Pseudo-instruction VMUL.F32 s1,s1,s0 ; r0 = radius VMUL.F32 s0,s1,s0 ; return area in s0 BX LR ENDP</pre>

A C function can call an assembly floating-point function, as shown below.

Main Program	Assembly Function
<pre>extern float sum(float *a, int size); int main(void){ float b = sum(a, 5); while(1); }</pre>	<pre>AREA fun, CODE, READONLY, ALIGN=3 EXPORT sum ; make sum visible sum PROC VLDR.F32 s0, =0.0 ; s = 0.0 MOVS r3, #0 ; i = 0 B check loop VLDR s1, [r0], #4 ; post-index VADD.F32 s0, s0, s1 ; s += a[i] ADDS r3, r3, #1 ; i++ check CMP r3, r1 BLT loop BX lr ; return ENDP END</pre>

The multiply-accumulate (MAC) instructions compute expression $\pm a \pm b \times c$ efficiently in one instruction. The following table lists all MAC instructions.

VLMA.F32 {Sd,} Sn, Sm	Multiply float, then accumulate float, $Sd = Sd + Sn * Sm$
VLMS.F32 {Sd,} Sn, Sm	Multiply float, then subtract float, $Sd = Sd - Sn * Sm$
VNMLA.F32 Sd, Sn, Sm	Multiply float, then accumulate, then negate float, $Sd = -1 * (Sd + Sn * Sm)$
VNMLS.F32 Sd, Sn, Sm	Multiply float, then subtract, then negate float, $Sd = -1 * (Sd - Sn * Sm)$

The following table lists fused MAC instructions.

VFMA.F32 {Sd,} Sn, Sm	Multiply (fused) then accumulate float, $Sd = Sd + Sn * Sm$
VFMS.F32 {Sd,} Sn, Sm	Multiply (fused) then subtract float, $Sd = Sd - Sn * Sm$
VFNMA.F32 {Sd,} Sn, Sm	Multiply (fused) then accumulate then negate float, $Sd = -1 * Sd + Sn * Sm$
VFNMS.F32 {Sd,} Sn, Sm	Multiply (fused) then subtract then negate float, $Sd = -1 * Sd - Sn * Sm$

A fused operation performs no intermediate rounding. For example, the floating-point fused multiply-add instruction “VLMA.F32 Sd, Sn, Sm” calculates “ $Sd + Sn * Sm$ ” as if with unlimited range and precision, and the rounding is only performed once at the end. A fused operation reduces rounding error but runs slower.

12.4.6 Single-precision comparisons

The comparison instructions, as shown below, compare two floating-point values and set the N, Z, C, and V bit flags of the Floating-point Status and Control Register (FPSCR). “MVRS APSR_nzcv FPSCR” copies the current N, Z, C, and V flag bits from the FPU’s FPSCR register to the processor’s APSR register.

VCMP.F32 Sd, <Sm #0.0>	Compare two floating-point registers, or one floating-point register and zero
VCMPE.F32 Sd, <Sm #0.0>	Compare two floating-point registers, or one floating-point register and zero, and raise exception for a signaling NaN

The following program shows a simple subroutine that returns the maximum of two floating-point values. Following the EABI standard, the floating-point input arguments come in via register s0 and s1, and the result comes out via register s0. The VCMPE instruction updates the NZCV flags in FPSCR. However, conditional execution does not check FPSCR. Therefore, we need to copy these flags to APSR.

C Program	Assembly Program
<pre>float max(float a, float b){ if (a > b) return a; else return b; }</pre>	<pre>max PROC EXPORT max VCMPE.F32 s0, s1 ; Copy NZCV flags to APSR VMRS APSR_nzcv, FPSCR BGT exit VMOV.F32 s0, s1 exit BX lr ENDP</pre>

Example 12-12. Selecting the max between two single-precision floating point numbers

The following gives two different assembly functions of finding the absolute value.

C Program	Assembly Program 1	Assembly Program 2
<pre>float fabs(float f) { if (f < 0) return -1.0*f; else return f; }</pre>	<pre>fabs PROC VCMPE.F32 s0, #0.0 ; Copy NZCV flags to APSR VMRS APSR_nzcv, FPSCR BCS exit VNEG.F32 s0, s0 exit BX lr ENDP</pre>	<pre>fabs PROC VABS.F32 s0, s0 BX lr ENDP</pre>

Example 12-13. Calculating the absolute value of a single-precision floating point number

12.4.7 Precision Conversion

The following table lists all instructions that convert values to a different format. The option “R” forces the operation to use the rounding mode specified by FPSCR. Otherwise, rounding towards zero is used.

VCVT{R}.S32.F32 Sd, Sm	Convert from single-precision float to signed 32-bit (S32) or unsigned 32-bit (U32) integer.
VCVT{R}.U32.F32 Sd, Sm	
VCVT{R}.F32.S32 Sd, Sm	Convert to single-precision float from signed 32-bit (S32) or unsigned 32-bit (U32) integer.
VCVT{R}.F32.U32 Sd, Sm	
VCVT{R}.Td.F32 Sd, Sm, #n	Convert between single-precision and fixed-point. Td can be S16 (signed 16-bit), U16 (unsigned 16-bit), S32 (signed 32-bit), and U32 (unsigned 32-bit). #n is the number of fraction bits in the fixed-point number.
VCVT{R}.Td.F32 Sd, Sd, #n	
VCVT{R}.F32.Td Sd, Sm, #n	
VCVT{R}.F32.Td Sd, Sd, #n	
VCVT<B T>.F32.F16 Sd, Sm	Converts half-precision float to single-precision (B = bottom half of Sm, T = top half of Sm)
VCVT<B T>.F16.F32 Sd, Sm	Converts single-precision float to half-precision (B = bottom half of Sd, T = top half of Sd)

Example 12-14 illustrates the assembly implementation of converting an integer to a single-precision floating point format with and without FPU.

With FPU	Without FPU
VMOV.F32 s1,#3 VCVT.F32.S32 s0,s0	MOV r0, #3 BL __aeabi_i2f __aeabi_i2f PROC ANDS r1,r0,#0x80000000 ; r[31] = Sign RSBSMI r0,r0,#0 ; If r0 < 0, r0 = 0 - r0 CLZ r3,r0 ; Count Leading zeros LSLS r2,r0,r3 ; Remove Leading zeros RSB r3,r3,#157 ; exp = 157 - # Leading zeros ; Note that it is not 158, ; extra 1 is added later BEQ exit ; if r2 is 0, return 0 ORRS r1,r1,r3,LSL #23 ; r1[30:23] = Exponent ADD r0,r1,r2,LSR #8 ; r0[22:0] = Significand ; Exponent = Exponent + 1 ; Round to Nearest LSLS r3,r2,#25 ; carry = round bit ; r3 holds sticky bits BXCC lr ; round down if round bit is 0 ADD r0,r0,#1 ; round up since round bit is 1 BICEQ r0,r0,#1 ; if sticky bits are 0 exit BX lr ENDP

Example 12-14. Converting integer to single-precision float

We will give two examples to explain the basic procedures of converting an integer to a single-precision floating point number without using FPU.

Example 1: Converting 7 to single-precision float

- 1) Input: r0 = 00000000_00000000_00000000_00000111
- 2) Obtain sign: r1 = 00000000_00000000_00000000_00000000
- 3) Count the leading zeros of input: r3 = 29
- 4) Remove leading zeros: r2 = 11100000_00000000_00000000_00000000
- 5) Bias = 127
- 6) Obtain exponent_minus_1:
r3 = 157 - 29 = 128 = 00000000_00000000_00000000_10000000
- 7) Place exponent: r1 = 00100000_00000000_00000000_00000000
- 8) Obtain significant: r2 LSR #8 = 00000000_11100000_00000000_00000000
- 9) Combine sign, exponent, and significant:
r1 + r2 LSR #8 = 01000000_11100000_00000000_00000000

Example 2: Converting -3 to single-precision float

- 1) Input: $r0 = 11111111_11111111_11111111_11111101$
- 2) Obtain sign: $r1 = 10000000_00000000_00000000_00000000$
- 3) Obtain absolute value: $r0 = 00000000_00000000_00000000_00000011$
- 4) Count the leading zeros of input: $r3 = 30$
- 5) Remove leading zeros: $r2 = 11000000_00000000_00000000_00000000$
- 6) Bias = 127
- 7) Obtain exponent_minus_1:
 $r3 = 157 - 30 = 127 = 00000000_00000000_00000000_01111111$
- 8) Place exponent: $r1 = 10111111_10000000_00000000_00000000$
- 9) Obtain significant: $r2 \text{ LSR } \#8 = 00000000_11000000_00000000_00000000$
- 10) Combine sign, exponent, and significant:
 $r1 + r2 \text{ LSR } \#8 = 11100000_01000000_00000000_00000000$

12.4.8 FPU Exception and Exception handling

As introduced previously, there are six floating-point exceptions, including input denormal (IDC), inexact (IXC), underflow (UFC), overflow (OFC), divide by zero (DZC), and invalid operation (IOC).

The following gives example instructions that set the exception flags in the FPSCR register.

```

VLD R.F32  s0, =0.0      ; Pseudo-instruction
VLD R.F32  s1, =-1.0     ; Pseudo-instruction
VDIV.F32   s2, s1, s0     ; s2 = -Infinity, Set DZC flag in FPSCR
VDIV.F32   s3, s0, s0     ; s3 = Quiet NaN (QNaN), Set IOC flag in FPSCR
VSQRT.F32  s4, s1        ; s4 = QNaN, Set IOC flag in FPSCR

VLD R.F32  s5, =2.12e30   ; A large constant,  $2.12 \times 10^{30}$ 
VMUL.F32   s6, s5, s5     ; Set OFC and IXC flags in FPSCR

VLD R.F32  s5, =2.12e-30  ; A large constant,  $2.12 \times 10^{-30}$ 
VMUL.F32   s6, s5, s5     ; Set UFC and IXC flags in FPSCR

```

By default, most processors do not handle these exceptions proactively. They do nothing more than setting exception flags to indicate an exception has occurred. This allows the computation to continue with a default output value specified by the IEEE 754 standard.

For example, the default output of the overflow, underflow, and invalid operator is infinity, zero, and NaN, respectively. Floating-point operations can propagate the infinity and NaN consistently throughout the computation. However, such default outputs may introduce devastating inaccuracies in some applications.

We can change the default output of floating-point exceptions. In general, a processor can handle an exception in two approaches: a *trap* or an *interrupt*.

- A *trap* handler returns an alternate result, instead of an exceptional result, and allows the program to resume execution.
- An *interrupt* handler takes over the control and performs alternate calculation when an exception occurs.

However, Cortex-M4 processors do not support traps. All floating-point exceptions are handled through via interrupts. As shown in Figure 12-22, five exceptions, including UFC, OFC, IDC, DZC, and IOC, are connected via a logical OR operator to generate FPU interrupts. Cortex-M4 processors do not allow the Inexact Exception (IXC) to produce FPU interrupts because rounding errors occur very frequently. Some processors allow the enabling and disabling the interrupt for each individual FPU exception.

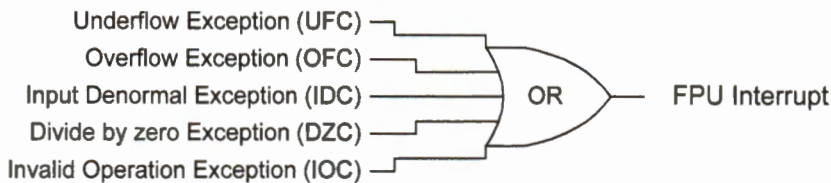


Figure 12-22. Cortex-M4 handles all floating-point exceptions via interrupts

The following assembly program shows how to process the overflow exception (OFC) in the FPU interrupt handler. Various methods are available to handle overflow. For example, the interrupt handler can convert the single-precision operation to a double-precision one to extend the range, or the handler can multiply the operands by a proper scaling factor to make the result within the current range. Software should enable FPU interrupts in NVIC by running “NVIC_EnableIRQ(FPU_IRQn);”.

```

FPU_IRQHandler PROC
    EXPORT FPU_IRQHandler

    VMRS r0, FPSCR      ; Read current FPSCR

    ; Overflow cumulative exception (OFC)
check_OFC ANDS r0, #(1<<2)
          BEQ exit

    ; Handle OFC exception
    ...

    ; Clear OFC flag
    ...

exit      BX LR
          ENDP
  
```

Clearing the FPU exception flags depends on the context save-and-restore behavior setting in the Floating-point Context Control Register (FPCCR) register. The following shows that example assembly code that clears all FPU exception flags.

1. Lazy save/store (default): LSPEN = 1, ASPEN = x

```
VMRS r0, FPSCR           ; Dummy access to force context saving
LDR r1, =FPU_BASE        ; Load the base memory address of FPU
LDR r0, [r1, #FPU_FPCAR] ; r0 = FPU->FPCAR
LDR r2, [r0, #64]        ; stacking 16 registers (s0-s15) and FPSCR
BIC r2, r2, #0x8F        ; Clear all flags
STR r2, [r0, #64]        ; Save into stack
                          ; When an interrupt exits, FPSCR is
                          ; updated during automatic unstacking
```

2. No FPU register saving: LSPEN = 0, ASPEN = 0

```
VMRS r0, FPSCR           ; Read FPSCR
BIC r0, r0, #8F          ; Clear all flags
VMSR FPSCR, r0           ; Save FPSCR
```

3. Automatic save/restore: LSPEN = 0, ASPEN = 1

```
; Clear FPSCR flags by using a dummy read
VMRS r1, FPSCR           ; Automatically cleared after read

; Clear exception flags saved in stack
TST lr, #4
MRSEQ r0, msp            ; If LR = 0xFFFFFFFF, SP = MSP
MRSNE r0, psp            ; If LR = 0xFFFFFFF, SP = PSP
LDR r2, [r0, #96]        ; FPU stack = 16 registers(s0-s15) + FPSCR
BIC r2, r2, #0x8F        ; Clear all flags
STR r2, [r0, #96]        ; Save into stack
```

12.4.9 Example Assembly Programs

12.4.9.1 Look up a Float Array

Example 12-15 shows a simple subroutine that looks up a specific float number in a float array. The C subroutine takes three arguments, the array memory address, a target float number, and the array size. If the subroutine finds the target float in the array, it returns its location index; otherwise, it returns -1.

The subroutine takes three arguments, including the float array address, the target float, and the array size, in register r0, s0, and r1, respectively. Note that the float array address is a 32-bit integer, and thus it is passed via a core register instead of a floating-point register.

The VCMF.F32 instruction compares two float numbers and sets up the NZCV flags in the FPSCR register based on the comparison result. We need to copy these flags to the APSR register for the subsequent conditional branch or conditionally executed instructions.

C Function	Assembly Function
<pre> int32_t lookup(float *fArray, float f, uint32_t size){ int i; for (i = 0; i < size; i++) { if (fArray[i] == f) return i; } return -1; } </pre>	<pre> lookup PROC EXPORT lookup ; Make lookup visible MOV r2, r0 ; r0 = array address MOV r3, #0 ; loop index i = 0 B check loop ; Find address of fArray[i] ADD r0, r2, r3, LSL #2 VLDR s1, [r0, #0] ; s1 = fArray[i] ; Compare f and fArray[i] VCMF.F32 s1, s0 VMRS APSR_nzcv, FPSCR ; copy flags ; Return index i if equal MOVEQ r0, r3 BXEQ lr ADDS r3, r3, #1 ; loop index ++ check CMP r3, r1 ; r1 = array size BCC loop MOV r0, #0xFFFFFFFF BX LR ; return -1 if failed ENDP </pre>

Example 12-15. Looking up a single-precision floating point array

12.4.9.2 Sine Function of Argument in Radians

Example 12-16 shows the C and assembly function that calculates the value of $\sin(x)$ based on the following *Taylor* expansion. The input x is in radians, not in degrees.

$$\sin(x) = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} x^{k+1} \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

The computation loop repeats until the program achieves the desired maximum error. The input x is passed via register $s0$, and the result is returned also in register $s0$.

According to EABI, a subroutine must preserve in the stack any single-precision registers $s16$ - $s31$ (double-precision registers $d8$ - $d15$) that are written in the subroutine. It is not required to preserve registers $s0$ - $s15$ ($d0$ - $d7$) across subroutine calls. In this example, the instruction “VPUSH.32 {s16-s21}” is equivalent to “VPUSH.64 {d8-d10}”. When multiple registers are pushed onto or popped off the stack, similar to PUSH and POP (see

Example 8-3), VPUSH pushes the largest numbered register first, and VPOP pops the smallest numbered register first. However, different from PUSH and POP, VPUSH and VPOP require that registers must be consecutive in their names.

C Function	Assembly Function
<pre>float sine(float x) { float ret = 0.0; float pow = x; float term = pow; float sign = 1.0; float fact = 1.0; int k = 1; while(fabs(term) >= 0.00001f){ term = sign * pow/fact; ret += term; pow *= x * x; fact *= 2*k * (2*k+1); sign = 0 - sign; k++; } return ret; }</pre>	<pre>sine PROC EXPORT sine PUSH {r4, lr} VPUSH.32 {s16-s21} VMOV.F32 s16, s0 ; s16 = x VLDR.F32 s18, =0 ; s18 = ret = 0.0 VMOV.F32 s17, s16 ; s17 = pow = x VMOV.F32 s19, s16 ; s19 = term = x VMOV.F32 s20, #1 ; s20 = sign = 1 VMOV.F32 s21, #1 ; s21 = fact = 1 MOVS r4, #1 ; r4 = k = 1 B check loop VMUL.F32 s0,s20,s17 ; s0 = sign * power VDIV.F32 s19,s0,s21 ; term = s0/fact VADD.F32 s18,s18,s19 ; ret += term VMUL.F32 s0,s16,s16 ; s0 = x*x VMUL.F32 s17,s0,s17 ; pow *= x*x LSLS r0, r4, #1 ; r0 = 2*k ADDS r1, r0, #1 ; r1 = 2*k + 1 MULS r0, r1, r0 ; r0 = 2*k*(2*k+1) VMOV s0, r0 VCVT.F32.S32 s0, s0 ; convert to float VMUL.F32 s21,s0,s21 ; fact *= 2k*(2k+1) VLDR.F32 s0, =0 VSUB.F32 s20,s0,s20 ; sign = 0 - sign ADDS r4, r4, #1 ; k++ check VABS.F32 s0, s19 VLDR.F32 s1, =0.00001 VCMPE.F32 s0, s1 VMRS APSR_nzcv, FPSCR ; copy NZCV flags BGE loop VMOV.F32 s0, s18 ; return ret in s0 VPOP.32 {s16-s21} POP {r4, pc} ENDP</pre>

Example 12-16. Calculating *sine* value based on Taylor expansion

12.5 Exercises

1. Suppose two real numbers A and B are in the UQ16.16 format. Show the basic procedure to calculate the product C. $C = A \times B$. Note C is also in UQ16.16 format. List all key operations and write an assembly program to conduct the multiplication.
2. Convert 3.1415 to UQ8.8 format, and convert -3.1415 to Q7.8 format.
3. Convert the binary real number 1111.1101 to decimal.
4. Write an assembly program that calculates the sum of an array of UQ8.8 numbers.
5. Write an assembly program that calculates the sum of an array of UQ16.16 numbers.
6. Convert 3.1415 to IEEE 754 single-precision format.
7. Convert 0xC1F54000 to IEEE 754 single-precision decimal
8. Manually subtract the following two IEEE 754 single-precision numbers:

$$0xC1D15053 - 0xC3FD5053$$

9. Manually add the two IEEE 754 single-precision numbers given in the previous question.
10. Manually multiply the following IEEE 754 single-precision numbers.

$$0xC0D40000 \times 0x41C20000$$

11. Manually divide the following IEEE 754 single-precision numbers.

$$0xC0D40000 \div 0x41C20000$$

12. How many byte(s) boundary should a “float” type be aligned to in C?
13. Given the following C functions, what registers are used to pass these arguments? Which register holds the return value?

(1) float fun1(float a, float b, float c, uint8_t d, int32_t e);

(2) double fun2(float f1, double f2, float f3, double f4);

(3) uint16_t fun3(uint8_t a1, int32_t a2, float f1, float f2, float f3, double f4)

(4) float * fun4(float *f1, float *f2, double *f3, float f4, double f5);

14. How many bytes of the stack are needed to pass arguments when the following function is called?

```
(1) float fun1(float a, float b, float c, float d,
              float d, float e, float f);
```

```
(2) float fun2(uint8_t a1, float f1, uint16_t a2, uint8_t a3,
              float f2, int32_t a4, float f3, uint8_t a5,
              float f4, int32_t a6, uint8_t a7);
```

```
(3) float fun3(float f1, float f2, float f3, float f4,
              float f5, float f6, float f7, float f8,
              float f9, float f10, float f11, float f12,
              float f13, float f14, float f15, float f16,
              float f17, float f18, float f19, float f20, float f21);
```

```
(4) double fun4(double d1, double d2, double d3, double d4, double d5,
               double d6, double d7, double d8, double d9, double d10);
```

15. Write an assembly function that finds the maximum value in a single-precision floating-point array.

```
float maxf(float *array, int array_size)
```

16. Write an assembly function that calculates the exponential value e^x .

```
float expf(float x)
```

The function should be based on the following Taylor expansion.

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} \approx 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

17. Write an assembly function that calculates the cosine value $\cos(x)$.

```
float cos(float x)
```

The function should be based on the following Taylor expansion.

$$\cos(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k}}{(2k)!} \approx 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \frac{x^8}{8!} - \dots$$

18. When multiple registers are pushed onto or popped off the stack in one instruction, the register list is sorted. The largest numbered register is pushed first onto the stack, and the smallest numbered register is popped first. Assume stack pointer (sp) = 0x20006000, register Ri = i and Si = 1.0*i (r0 = 0, r1 = 1, ..., and s0 = 0.0,

$s1 = 1.0$, $s2 = 2.0$, ...). Show the stack memory content in hex immediately after the processor executes the following two instructions.

`push {r3-r5,r1,r9,r7}` ; Registers do not have to be listed consecutively
`vpush.32 {s16-s18}` ; Registers in the list must be consecutive

19. Translate the following assembly function into a C function. Explain what this function does and give the function a meaningful name.

```
myfun PROC
    EXPORT myfun
    VMUL.F32  s0, s0, s0
    VLMA.F32  s0, s1, s1
    VLMA.F32  s0, s2, s2
    VSQRT.F32 s0, s0
    BX lr
ENDP
```

20. Assume the main stack (MSP) is used.

- Stack pointer (sp) = 0x20006000, Register R1 = i and S1 = 1.0*i (r0 = 0, r1 = 1, ..., and s0 = 0.0, s1 = 1.0, s2 = 2.0, ...)
- Additionally, the program status register (PSR) = 0x00000020, PC = 0x08000020, LR = 0x20008020, and FPCAR = 0x03000000.

Show the stack memory content in hex immediate after an interrupt service routine is called in the following scenarios.

- (1) There is no FPU or FPU has not been used.
- (2) FPU has been used before the interrupt occurs, and the interrupt service routine will run floating-point instructions.

21. What is the value in register S1 (i = 0, 1, ..., 31) after the following SysTick interrupt handler has been executed once? Assume initially S1 = 1.0*i (s0 = 0.0, s1 = 1.0, s2 = 2.0, ...).

```
SysTick_Handler PROC
    VPUSH {s17-s19}
    VMOV.F32 s3, #1
    VADD.F32 s0, s3, s1
    VADD.F32 s4, s3, s1
    VADD.F32 s13, s3, s1
    VADD.F32 s15, s10, s1
    VADD.F32 s18, s15, s1
    VADD.F32 s20, s7, s1
    VADD.F32 s31, s10, s12
    VPOP {s17-s19}
    BX LR
ENDP
```

22. Suppose the SysTick interrupt occurs when

- PC = 0x08000044, XPSR = 0x00000020,
- SP = 0x20000200, LR = 0x08001000,
- FPCAR = 0x03000000,
- Processor's Register $R_i = i$, $i = 0, 1, 2, \dots$, and
- FPU Registers $S_i = 1.0 * i$ ($s_0 = 0.0$, $s_1 = 1.0$, $s_2 = 2.0$, ...).

Memory Address	Instruction
	_main PROC
	...
0x08000044	VMOV.F32 s3, #2
	...
	ENDP
	SysTick_Handler PROC
	EXPORT SysTick_Handler
0x0800001C	VADD.F32 s15, s15, s11
0x08000020	VADD.F32 s17, s16, s10
0x08000024	BX lr
	ENDP

- (1) Show the stack contents and the value of PC and SP when immediately entering the SysTick interrupt service routine.
- (2) What is the value in register s15 and s17?